चाणक्य

Chanakya

How it was built, and what every part does

A plain-English guide for readers with no technical background

PRODUCT	Chanakya v1, a personal planner
OWNER	Anshuman Tewary
LIVE AT	chanakya-seven-sigma.vercel.app
BUILT	8–9 November 2026

Contents

1. **Part 1: The story**
    1. What Chanakya is
    2. Words you'll meet
    3. The tools, and why each was chosen
    4. How it was built, step by step
    5. What happens when someone uses it
    6. How "Plan my day" decides
    7. Where the information lives, and how it's kept private
    8. Keys and secrets
    9. How it was tested
    10. Accounts, costs and what's left to do
2. **Part 2: Every file, explained**
    1. How to read code (two pages)
    2. Map of the project folder
    3. The project's setup files
    4. The database
    5. The behind-the-scenes helpers
    6. The look and the shared frame
    7. The public pages
    8. Onboarding
    9. The signed-in app

> Part 1 can be read on its own. Part 2 shows every line of the code. Each small block of lines has a plain-English explanation beside it. You never need to understand the code itself; the explanations are written to be read on their own.

Part 1: The story

1. What Chanakya is

Chanakya is a personal planner that runs in any web browser, on a phone or a computer. Its job is to answer one question: *"I have all these things to do. When should I do each one?"*

The idea came from a simple problem. People have too much to do and rarely know when to do it. When free time appears they feel overwhelmed and don't know where to start, and their to-dos are scattered across calendars, notes apps, spreadsheets and WhatsApp messages.

Version 1 does five things:

1. **Knows your day.** A short form asks when you wake up, when you sleep, and how many hours you can give to tasks.
2. **Collects your tasks.** Each task has a name, a priority (high, medium or low), how long it takes, and optionally a deadline. Tasks can repeat daily or weekly.
3. **Plans your day.** One button, "Plan my day", places your tasks into your free hours, most important first, with short breaks between.
4. **Tracks what you did.** Each task can be marked done, postponed or cancelled, and you can move or remove things if the plan doesn't suit you.
5. **Shows your progress.** A dashboard shows how much of today's plan you completed, your streak of active days, and the last 7 days.

Everyone logs in with their own email and password and only ever sees their own information. The full requirements live in the project as docs/PRD.md (a "product requirements document").

Deliberately left out of version 1: AI features, money and health tracking, notifications, calendar connections, teams, and a "daily wisdom" card (decided on 8 November to keep version 1 focused).

2. Words you'll meet

Code	Written instructions for a computer. Each file in the project is a set of these instructions.
Website / web app	A website you can use, not just read. Chanakya is a web app: it stores your tasks and responds to your clicks.
Browser	The program you use to visit websites: Chrome, Edge, Safari.
Server	A computer somewhere else that builds pages and answers requests. When you open Chanakya, a server in Mumbai prepares your page and sends it to your browser.
Database	An organised store of information, like a set of spreadsheets. Chanakya's holds profiles, tasks and plans.
Framework	A ready-made skeleton for building a certain kind of software, so you don't start from nothing. Chanakya uses Next.js.
Package / library	A ready-made building block written by someone else, downloaded and reused (for example, the piece that talks to Supabase).
Deploy / publish	Putting the latest version of the app on the internet so others can use it.

Git and commits	Git is a save-history tool. A *commit* is one saved snapshot of the whole project with a short note, so any earlier version can be brought back.
Keys	Long codes that let the app use an online service, like a key card. Some are fine to be public; others must stay secret (see section 8).
Environment variables	Settings, such as keys, kept outside the code in a separate file (.env.local) or in the hosting service's settings, so secrets are never saved with the code.
Login cookie	A small note your browser keeps after you log in, so the site remembers you between pages.
Front end / back end	The front end is what you see and click. The back end is the behind-the-scenes work: checking who you are, saving data.
Terminal / PowerShell	A window where you type commands instead of clicking. PowerShell is the Windows version.

3. The tools, and why each was chosen

Tool	What it is, in plain words	Why it was used
Node.js and npm	Node.js runs this kind of code on a computer. npm downloads building blocks.	Needed to create, run and build the app.
Next.js (v16)	The framework: the skeleton of the website.	Popular, well supported, and made by Vercel, so hosting is smooth.
React	The part that draws screens from small reusable pieces.	Comes with Next.js.
TypeScript	A spell-checker for code that catches mistakes before the app runs.	Fewer bugs reach real users.
Tailwind CSS	A styling system: short words like "rounded" or "bg-saffron" describe appearance.	Fast to style and keeps the look consistent.
ESLint	A proof-reader for code.	Flags risky patterns.
Supabase	An online service providing logins and a database.	Handles passwords and data safely without running our own servers. Free plan. Project in Mumbai.
Vercel	A hosting service that puts the app on the internet.	Made for Next.js. Free plan. Server set to Mumbai.
Git	The save-history tool.	Every step is saved and can be undone.
PostHog	An analytics service that counts how people use the app.	To measure the PRD's success numbers. Built in, waiting for a key.
VS Code and **Claude Code**	VS Code is the editor where files are opened. Claude Code is the AI assistant that wrote and tested the code inside it.	The working environment.

4. How it was built, step by step

Everything below happened over two days, 8 and 9 November 2026, with Anshuman directing and Claude Code doing the hands-on work. Each finished step was saved as a git commit.

Step 1: Create an empty app. A standard Next.js starter project was created in a folder called chanakya, using TypeScript, Tailwind, ESLint, the "App Router" page system, the @/ shortcut and npm. It was started on the computer and opened at localhost:3000 (the address for "this computer").

Step 2: First change. The starter's heading was replaced with "Welcome to Chanakya" in app/page.tsx. (The first attempt didn't save, so Claude made the edit directly.)

Step 3: Start saving history. Git was set up and the first snapshot saved. The starter's logos, links and pictures were then removed and the browser-tab title set to "Chanakya". commits 1–2

Step 4: The requirements. Anshuman supplied the PRD, saved as docs/PRD.md. It set the features, the data to store, the success measures and a launch date of Saturday 14 November.

Step 5: The planning brain first. lib/planner.ts, the "Plan my day" rules, was written and tested with made-up tasks. The test caught a real mistake: weekly tasks were being treated as overdue. It was fixed and re-tested.

Step 6: The database design. supabase/schema.sql describes the three tables, the privacy locks and the "delete my account" action. Two extra fields were added beyond the PRD: each person's time zone (so "today" is *their* today) and an "edited" marker (to see how often plans get changed by hand, as the PRD's risks section asks).

Step 7: All the screens. Login, onboarding, Tasks, Today and Dashboard, plus the doorman (proxy.ts) and every button's server action. The rules for this new version of Next.js were read first: personal parts of a page must sit in a "loading area". A build error (the "keys missing" check ran too early) was found and fixed. commit 3

Step 8: Supabase set up. Anshuman signed in to Supabase and created a temporary access key. Claude used it to create the **chanakya** project in Mumbai with a strong random database password (not the phone-number password offered, which would be easy to guess), create the tables, switch on the privacy locks and set the login addresses. A safety check stopped Claude from turning off email confirmation, so that decision stays with the owner.

Step 9: Testing with real data. Two temporary test accounts proved that each person's data is private. Then a full run-through of the app (sign in, onboarding, add tasks, plan, mark done, re-plan, dashboard, sign out) passed. All test accounts were deleted.

Step 10: Going live. Anshuman approved Vercel on this computer. Claude created the Vercel project, gave it only the two public Supabase values (and caught and fixed invisible stray characters in them), set the server to Mumbai, published the site, told Supabase the new address, and re-ran the full test on the live site: all passed. commit 4

Step 11: Decisions. The wisdom card was taken out of version 1 and recorded in the PRD. The long-term vision document was set aside for later. commit 5

Step 12: Analytics. PostHog tracking was added for every success measure in the PRD. It was tested with a stand-in service on this computer, and it stays switched off until a PostHog key is added. The starter README was replaced with a Chanakya one. commit 6

Step 13: The ancient Indian look. New colours (parchment, saffron, maroon, temple gold, with a lamp-lit night mode), Indian typefaces, the चाणक्य logo, a mandala, lotus, temple-border bands and a fuller landing page. Screenshots were checked in day and night modes and at phone size, then the redesign was published. commit 7

Step 14: Tidying and this guide. A duplicated rule in the "never save" list was cleaned up, and this guide was written. commits 8–9

5. What happens when someone uses it

Visitor opens the web address → **Doorman** (proxy.ts) checks the login → Not logged in: **Login page** →
First time: **Onboarding form** → **Today**: Plan my day → Done / Postpone / Cancel →
Dashboard: progress and streak

Behind each click, the same pattern repeats:

1. Your browser sends the click to the server in Mumbai.
2. The server checks who you are, using your login cookie and Supabase.
3. It reads or saves information in the database, which only allows your own rows.
4. It sends back the updated page. If analytics is switched on, an anonymous event is sent to PostHog *after* the page is on its way, so it never slows you down.

6. How "Plan my day" decides

Which tasks are considered: daily tasks every day; weekly tasks on their chosen weekday; one-time tasks until they're done; deleted tasks never. Tasks you've already dealt with today, or moved by hand, are left where they are.

The order:

1. Tasks due today or overdue go first.
2. Then higher priority before lower.
3. Then the nearest deadline.
4. Then shorter tasks first, to build momentum.

Placing them: starting from your wake-up time, or from right now if that's later, each task goes into the next free slot, followed by a **10-minute break**. It stops at bedtime or when your daily hours are used up. A task too big to fit goes on a "Not in today's plan" list, and smaller tasks after it can still use the remaining time.

Worked example (from the live test). Wake 06:00, sleep 23:30, 6 hours available, planned at about 09:20:

Task	Details	Result
Write weekly report	High priority, 60 min	09:20–10:20
Gym	Medium, 45 min, daily	10:30–11:15
Read a chapter	Low, 30 min	11:25–11:55
Plan next quarter	High, 600 min	Didn't fit (more than the 6 hours available)

7. Where the information lives, and how it's kept private

All information is stored in Supabase's database in Mumbai, in three tables:

Table	One row is...	Holds
profiles	one person	user type, wake time, sleep time, hours per day, time zone
tasks	one task	name, priority, minutes, deadline, repeat rule, whether it's deleted
plan_items	one task placed on one day	date, start, end, status, whether it was moved by hand

The privacy lock. Picture every row in a locked drawer with its owner's name on it. The database itself, not just the app, refuses to open a drawer for anyone but its owner. This is called Row Level Security. It was tested: a second account, and a visitor who wasn't logged in, could see nothing and change nothing.

Deleting your account (Dashboard → Settings) permanently removes the account and everything belonging to it.

Passwords are handled entirely by Supabase and are never visible to the app or to its owner.

8. Keys and secrets

Key	Public or secret?	Where it is kept
Supabase project address and *publishable* key	Public by design: the privacy locks protect the data	.env.local on this computer, and Vercel's settings
Supabase database password	**Secret**	Only in .env.local on this computer. Never uploaded.
Supabase secret / service_role key	**Secret**: full control	Not stored anywhere in the project. Only used briefly for testing.
Supabase personal access key (sbp_...)	**Secret**: controls the whole account	Used for set-up, then **deleted by Anshuman**, as recommended.
PostHog project key (phc_...)	Public by design	Not added yet.

The file .env.local is on git's "never save" list, so keys can't be saved into the project history by accident.

9. How it was tested

- **Code checks** on every change: TypeScript (mistakes), ESLint (risky patterns) and a full production build (the same build Vercel runs).
- **Planner tests:** ordering, breaks, starting from the current time, stopping at bedtime, bedtimes after midnight, avoiding busy time, and the daily-hours limit.
- **Privacy tests** against the real database, using two temporary accounts.
- **Full run-through tests** that use the real app like a browser would. They ran on this computer and twice on the live site.
- **Analytics test** with a stand-in PostHog, which confirmed the right events arrive, with no emails or task names in them.
- **Visual checks** of screenshots in day and night modes and at phone width.

Honest note: the redesign (step 13) only changed appearance. Because the Supabase access key had been deleted by then, the full run-through couldn't create a test account for that final check. The public pages and the login redirect were checked on the live site, and a manual click-through by the owner is recommended.

10. Accounts, costs and what's left to do

Service	Account	Plan
Supabase	Signed in with GitHub, organisation "Chanakya", project "chanakya" (Mumbai)	Free
Vercel	Signed in with GitHub, project "chanakya" (server in Mumbai)	Free (Hobby)
PostHog	Not created yet	Free

Still to do before launch (Saturday 14 November):

1. **PostHog:** sign up, then add the project key so analytics switches on.
2. **Email confirmation:** decide whether new users must click an emailed link before signing in. Supabase's free plan only sends a few emails an hour, which 5 testers could hit.
3. **Click through the app** once yourself, ideally on a phone.
4. **Send the link** to the 5 testers and collect feedback.
5. *Optional:* delete the unused Supabase project in Tokyo; connect GitHub to Vercel for automatic publishing; choose a nicer web address.

How to update the live site later: after a change, run `npx vercel@latest deploy --prod` in the project folder.

Part 2: Every file, explained

How to read code (two pages)

You don't need to understand the code to follow the explanations. But a few patterns appear everywhere, and knowing them makes the pages less mysterious.

You see	It means
`// a note` or `/* a note */` or `# a note` or `-- a note`	A comment: a note for people, ignored by the computer.
`import { thing } from "place"`	"Borrow *thing* from another file or package." Almost every file starts with these lines.
`export`	"Other files may borrow this."
`const name = …` / `let name = …`	Give a value a name, so it can be used later. `const` never changes; `let` can.
`function name(…) { … }`	A recipe: a named set of steps, which can be given ingredients (inside the round brackets).
`async` / `await`	"This step involves waiting (for example, for the database). Wait for the answer before continuing."
`if (…) { … } else { … }`	"If this is true, do the first part; otherwise do the second."
`return …`	"The recipe is finished; here is the result."
`{ … }`	Curly braces group things: the steps of a recipe, or a bundle of named values.
`[ … ]`	A list.
`"text"`	Words used as words (not as instructions).
`===`, `!==`, `&&`, `||`, `!`	"Is exactly equal to", "is not equal to", "and", "or", "not".
`type Something = …`	A description of what a piece of data looks like (TypeScript only).
`<div> … </div>`, `<button>`, `<h1>`	Screen descriptions, written like HTML, the language of web pages. A `<button>` is a button; `<h1>` is a main heading.
`className="…"`	How it should look: Tailwind words such as `rounded-lg` (rounded corners) or `text-maroon` (maroon text).
`"use client"` / `"use server"`	"This file runs in the visitor's browser" / "This file only ever runs on the server."

File endings: `.ts` is TypeScript code; `.tsx` is TypeScript that also describes screens; `.css` is styling; `.sql` is database instructions; `.json` and `.mjs` are settings.

Map of the project folder

```
chanakya/
├─ app/                    every page of the website
```

```
│  ├─ page.tsx              the landing page
│  ├─ layout.tsx            the outer frame for every page
│  ├─ globals.css           colours, background and decorations
│  ├─ error.tsx             the "something went wrong" page
│  ├─ login/                the login page, its form and what its buttons do
│  ├─ auth/callback/        where the "confirm your email" link lands
│  ├─ onboarding/           the "tell us about your day" form
│  └─ (app)/                the signed-in pages (the brackets don't appear in the address)
│     ├─ layout.tsx         the menu bar around them
│     ├─ actions.ts         what every button does
│     ├─ today/             the Today page
│     ├─ tasks/             the Tasks page and Edit task page
│     └─ dashboard/         the Dashboard
├─ components/ornaments.tsx the mandala, lotus, divider and logo
├─ lib/                     behind-the-scenes helpers
│  ├─ planner.ts            the "Plan my day" rules
│  ├─ session.ts            "who is logged in?"
│  ├─ supabase/server.ts    the connection to Supabase
│  ├─ analytics.ts          PostHog events
│  └─ time.ts, types.ts, ui.ts   dates, data shapes, shared styles
├─ supabase/schema.sql      creates the database tables and privacy locks
├─ proxy.ts                 the doorman that runs before every page
├─ docs/PRD.md              the product requirements
├─ README.md                a short guide for developers
├─ package.json             the project's ID card and shopping list
├─ next.config.ts, tsconfig.json, eslint.config.mjs   tool settings
├─ vercel.json              hosting settings (server in Mumbai)
├─ .env.example             empty template of the keys the app needs
├─ .gitignore               files git must never save
│
│  Not saved in git (created automatically, or private):
├─ .env.local               the real keys (private, this computer only)
├─ node_modules/            downloaded building blocks
└─ .next/                   Next.js's working files
```

> Not shown in Part 2: package-lock.json (an automatic record of the exact version of every downloaded building block, thousands of lines long), next-env.d.ts (generated automatically), app/favicon.ico (the small tab icon), and the two text documents README.md and docs/PRD.md, which are already plain English.

The project's setup files

These files don't show anything on screen. They tell the computer which tools the project uses and how to run them, a bit like the instruction sheet that comes with flat-pack furniture.

package.json

28 lines

The project's ID card and shopping list. It names the project, lists the ready-made building blocks ("packages") it needs, and defines short commands such as npm run dev.

```
{
  "name": "chanakya",
  "version": "0.1.0",
  "private": true,
```

The opening brace { starts the list. The project is called "chanakya", is at version 0.1.0, and is marked private so it can never be published to the public package store by accident.

```
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  },
```

Shortcuts. Typing npm run dev starts the app on your computer, npm run build makes the finished, optimised version, npm run start runs that finished version, and npm run lint checks the code for common mistakes.

```
  "dependencies": {
    "@supabase/ssr": "^0.12.7",
    "@supabase/supabase-js": "^2.119.0",
    "next": "16.4.5",
    "react": "19.3.0",
    "react-dom": "19.3.0"
  },
```

Building blocks the app needs to run. next is the framework (the skeleton of the website), react draws the screens, and the two @supabase packages let the app talk to the Supabase login and database service. The numbers are version numbers.

```
  "devDependencies": {
    "@tailwindcss/turbopack": "^4",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9",
    "eslint-config-next": "16.4.5",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```

Tools only needed while building, not by visitors: Tailwind (the styling system), TypeScript (a spell-checker for code), ESLint (a code-quality checker) and type descriptions that help TypeScript understand Node, React and React DOM.

next.config.ts

17 lines

Settings for Next.js, the framework that runs the website.

```
import type { NextConfig } from "next";

```

Line 1 borrows the description of what a valid settings object looks like, so mistakes are caught early.

```
const nextConfig: NextConfig = {
  /* config options here */
  cacheComponents: true,
  partialPrefetching: true,
  turbopack: {
    rules: {
      "*.css": {
        loaders: ["@tailwindcss/turbopack"],
        as: "*.css",
      },
    },
  },
};

export default nextConfig;
```

The settings themselves. cacheComponents and partialPrefetching are newer speed features: pages show their fixed parts instantly and fill in personal parts (like your tasks) a moment later. The turbopack rule tells the fast bundler to pass style files through Tailwind.

Hands these settings to Next.js.

tsconfig.json

34 lines

Settings for TypeScript, the checker that reads the code before it runs and points out mistakes such as a missing piece of information or a misspelt name.

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
```

compilerOptions are the checker's rules. strict: true turns on the strictest checking. noEmit means TypeScript only checks; Next.js does the actual building. jsx: react-jsx lets code contain HTML-like screen descriptions.

```
    "paths": {
      "@/*": ["./*"]
    }
  },
```

Creates the shortcut @/ meaning "the top of the project folder", so files can say @/lib/ui instead of a long ../../lib/ui.

```
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```

Which files to check (every .ts and .tsx file plus Next.js's generated descriptions) and which to skip (node_modules, the downloaded building blocks).

eslint.config.mjs

18 lines

Settings for ESLint, a proof-reader for code that flags risky or messy patterns.

```
import { defineConfig, globalIgnores } from "eslint/config";
import nextVitals from "eslint-config-next/core-web-vitals";
import nextTs from "eslint-config-next/typescript";

const eslintConfig = defineConfig([
  ...nextVitals,
  ...nextTs,
  // Override default ignores of eslint-config-next.
```

Loads Next.js's recommended proof-reading rules: one set for web performance and good practice, one for TypeScript.

```
  globalIgnores([
    // Default ignores of eslint-config-next:
    ".next/**",
    "out/**",
    "build/**",
    "next-env.d.ts",
  ]),
]);

export default eslintConfig;
```

Folders the proof-reader should skip because they are generated automatically (.next, out, build) rather than written by a person.

.gitignore

45 lines

A list of files git (the save-history tool) must never save. It keeps out bulky downloaded files and, most importantly, the private keys.

```
# See https://help.github.com/articles/ignoring-files/ for
more about ignoring files.

# dependencies
/node_modules
/.pnp
.pnp.*
.yarn/*
!.yarn/patches
!.yarn/plugins
!.yarn/releases
!.yarn/versions

```

Lines starting with # are notes for people. /node_modules is the large folder of downloaded building blocks; it can always be re-downloaded, so it isn't saved.

```
# testing
/coverage

# next.js
/.next/
/out/

# production
/build

```

Generated folders for test reports, Next.js's working files (.next) and finished builds: all recreated automatically.

```
# misc
.DS_Store
*.pem

# debug
npm-debug.log*
yarn-debug.log*
yarn-error.log*
.pnpm-debug.log*

```

System clutter (.DS_Store from Macs), certificate files (*.pem) and error logs.

```
# env files (can opt-in for committing if needed)
.env*
!.env.example

# vercel
.vercel

# typescript
*.tsbuildinfo
next-env.d.ts

# supabase cli
supabase/.temp/
```

.env* blocks every key file, including .env.local where the real keys live. The ! on the next line makes one exception: .env.example, the empty template, is allowed to be saved.

Vercel's link file, TypeScript's speed-up file, a generated Next.js file and a temporary Supabase folder: none of these belong in the saved history.

.env.example

8 lines

An empty template showing which keys the app needs. The real values go in .env.local on your computer and in Vercel's settings, never in saved code.

```
# Supabase keys: Supabase dashboard -> Connect -> App Frameworks -> Next.js
# Restart "npm run dev" after changing this file.

NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=

# PostHog analytics (optional): Project API key (phc_...) and host. Tracking is off when empty.
POSTHOG_KEY=
POSTHOG_HOST=https://us.i.posthog.com
```

Notes saying where to find the Supabase values, and that the app must be restarted after changing them.

The address of your Supabase project and its publishable key. Both are designed to be public: the database's privacy rules (see schema.sql) are what protect the data.

Optional analytics settings. While POSTHOG_KEY is empty, the app sends no analytics at all.

vercel.json

4 lines

Settings for Vercel, the service that hosts the live website.

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "regions": ["bom1"]
}
```

"regions": ["bom1"] runs the website's server in Mumbai, right next to the Supabase database (also in Mumbai), so each page needs only a short trip to fetch data.

The database

The database is where everything people type is stored. Think of it as a set of spreadsheets that live on Supabase's computers. This one file creates them.

supabase/schema.sql — 75 lines

Instructions, in the database language SQL, that create Chanakya's three tables, lock each person's rows away from everyone else, and add a "delete my account" action. It was run once on the Supabase project.

```
1   -- Chanakya v1 database (PRD section 8 and 11).
2   -- Paste this whole file into Supabase: SQL Editor -> New
query -> Run.
3
4   -- ---------- Tables ----------
5
```

Lines starting with -- are notes. They say what this file is and how to run it.

```
6   create table public.profiles (
7     user_id     uuid primary key references auth.users (id) on
delete cascade,
8     user_type   text not null check (user_type in ('student',
'professional', 'other')),
9     wake_time   time not null,
10    sleep_time  time not null,
11    daily_hours numeric(4, 1) not null check (daily_hours > 0
and daily_hours <= 24),
12    timezone    text not null default 'UTC', -- so "today" means
the user's today, not the server's
13    created_at  timestamptz not null default now()
14  );
15
```

The profiles table: one row per person with their answers from the onboarding form. user_id links to Supabase's list of accounts, and on delete cascade means "if the account is deleted, delete this row too". check (...) rules refuse nonsense, like a user type that isn't one of the three choices or more than 24 hours in a day. timezone remembers where the person lives, so "today" means their today.

```
16  create table public.tasks (
17    id               uuid primary key default gen_random_uuid(),
18    user_id          uuid not null default auth.uid() references
auth.users (id) on delete cascade,
19    title            text not null check (length(trim(title)) >
0),
20    priority         text not null default 'medium' check
(priority in ('high', 'medium', 'low')),
21    duration_minutes integer not null check (duration_minutes
between 5 and 720),
22    deadline         date,
23    repeat_rule      text not null default 'none' check
(repeat_rule in ('none', 'daily', 'weekly')),
24    active           boolean not null default true,
25    created_at       timestamptz not null default now()
26  );
27
```

The tasks table: every task anyone adds. Each gets a unique id automatically. user_id fills itself in with whoever is logged in. Rules make sure the title isn't blank, priority is high, medium or low, duration is between 5 and 720 minutes, and repeat is none, daily or weekly. active becomes false when a task is deleted, so past history stays correct.

```
28  create table public.plan_items (
29    id         uuid primary key default gen_random_uuid(),
30    user_id    uuid not null default auth.uid() references
auth.users (id) on delete cascade,
31    task_id    uuid not null references public.tasks (id) on
delete cascade,
32    date       date not null,
33    start_time time not null,
34    end_time   time not null,
35    status     text not null default 'planned' check (status in
('planned', 'done', 'postponed', 'cancelled')),
36    edited     boolean not null default false, -- PRD risk 1:
how often users change the plan
37    created_at timestamptz not null default now(),
38    unique (task_id, date)
39  );
40
```

The plan_items table: each row is one task placed at a time on one day by "Plan my day". status records what happened (planned, done, postponed or cancelled). edited notes whether the person moved it by hand, which helps answer the PRD's question "how often do plans feel wrong?". unique (task_id, date) stops the same task appearing twice on the same day.

```
  create index tasks_user_idx on public.tasks (user_id);
  create index plan_items_user_date_idx on public.plan_items
(user_id, date);

  -- ---------- Row Level Security: users only see their own
rows ----------

  alter table public.profiles   enable row level security;
  alter table public.tasks      enable row level security;
  alter table public.plan_items enable row level security;

  create policy "own profile" on public.profiles
    for all using (auth.uid() = user_id) with check (auth.uid()
= user_id);

  create policy "own tasks" on public.tasks
    for all using (auth.uid() = user_id) with check (auth.uid()
= user_id);

  create policy "own plan items" on public.plan_items
    for all using (auth.uid() = user_id) with check (
      auth.uid() = user_id
      and exists (select 1 from public.tasks t where t.id =
task_id and t.user_id = auth.uid())
    );

  -- ---------- "Delete my data" (PRD section 11) ----------
  -- Deleting the auth user cascades to every table above.

  create function public.delete_my_account()
  returns void
  language sql
  security definer
  set search_path = ''
  as $$
    delete from auth.users where id = auth.uid();
  $$;

  revoke all on function public.delete_my_account() from public,
anon;
  grant execute on function public.delete_my_account() to
authenticated;
```

Indexes are like the index at the back of a book: they let the database find one person's tasks, or one day's plan, quickly without reading every row.

Row Level Security (RLS) is the privacy lock. Turning it on means that by default nobody can see any row.

Policies are the exceptions to that lock: "you may read and change a row only if its user_id is yours" (auth.uid() means "the logged-in person"). The plan items rule also checks that the task belongs to you, so nobody can attach plans to someone else's task.

A ready-made action called delete_my_account. It deletes the logged-in person's account, and because every table uses on delete cascade, all their profile, tasks and plans disappear with it. The last two lines allow only logged-in people to use it.

The behind-the-scenes helpers

These files never draw anything on screen. They are the helpers that the screens rely on: checking who is logged in, talking to the database, working out dates, and planning the day.

proxy.ts

43 lines

A doorman that runs before every page. It keeps your login fresh and sends people to the right place: logged-out visitors to the login page, logged-in visitors past it.

```
1   import { createServerClient } from "@supabase/ssr";
2   import { NextResponse, type NextRequest } from "next/server";
3
```

Lines 1–2 bring in two helpers: one that creates a Supabase connection on the server, and Next.js's tools for reading a request and sending a response.

```
4   const PUBLIC_PATHS = ["/", "/login", "/auth/callback"];
5
6   // Runs before every page: keeps the login fresh and sends
people to the right place.
```

The pages anyone may see without logging in: the home page, the login page and the address the confirmation email link goes to.

```
7   export async function proxy(request: NextRequest) {
8     const url = process.env.NEXT_PUBLIC_SUPABASE_URL;
9     const key =
process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY;
10    if (!url || !key) return NextResponse.next(); // pages will
show a "keys missing" message
11
```

The doorman itself. It first reads the two Supabase keys; if they are missing it just lets the request through, so the page can show a helpful "keys missing" message.

```
12    let response = NextResponse.next({ request });
13    const supabase = createServerClient(url, key, {
14      cookies: {
15        async getAll() {
16          return request.cookies.getAll();
17        },
18        setAll(cookiesToSet, headers) {
19          cookiesToSet.forEach(({ name, value }) =>
request.cookies.set(name, value));
20          response = NextResponse.next({ request });
21          cookiesToSet.forEach(({ name, value, options }) =>
response.cookies.set(name, value, options));
22          Object.entries(headers ?? {}).forEach(([k, v]) =>
response.headers.set(k, v));
23        },
24      },
25    });
26
```

Prepares to let the visitor through, and creates a Supabase connection that reads the visitor's login cookie (a small note the browser keeps) and can write a refreshed one if the login is about to expire.

```
27    const { data } = await supabase.auth.getClaims();
28    const signedIn = Boolean(data?.claims?.sub);
29    const path = request.nextUrl.pathname;
30
```

Asks Supabase "is this login genuine, and whose is it?" signedIn becomes true or false, and path is the page being asked for.

```
31    let goTo: string | null = null;
32    if (!signedIn && !PUBLIC_PATHS.includes(path)) goTo =
"/login";
33    if (signedIn && (path === "/" || path === "/login")) goTo =
"/today";
34    if (!goTo) return response;
35
```

The decision. Not logged in and asking for a private page? Go to /login. Logged in and on the home or login page? Go straight to /today. Otherwise carry on.

```
36    const redirect = NextResponse.redirect(new URL(goTo,
request.url));
37    response.cookies.getAll().forEach((c) =>
redirect.cookies.set(c));
38    return redirect;
39  }
40
```

When redirecting, any refreshed login cookies are copied onto the redirect so the visitor stays logged in.

```
export const config = {
  matcher: ["/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp|ico)$).*)"],
};
```

Tells Next.js to run the doorman on every page except pictures and built-in files, which don't need checking.

lib/supabase/server.ts

35 lines

Creates a connection to Supabase that knows who the current visitor is. Every page and button that touches data uses it.

```
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

```

Brings in the Supabase connection maker and Next.js's way of reading cookies.

```
export function supabaseKeys() {
  const url = process.env.NEXT_PUBLIC_SUPABASE_URL;
  const key = process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY;
  if (!url || !key) return null;
  return { url, key };
}

/** Supabase client for Server Components, Server Actions and Route Handlers. */
```

Reads the project address and publishable key from the settings. If either is missing it returns null ("nothing").

```
export async function createClient() {
  // Read cookies first: this marks the page as "per visitor", so the build never runs the code below.
  const cookieStore = await cookies();
  const keys = supabaseKeys();
  if (!keys) {
    throw new Error(
      "Supabase keys are missing. Add NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY to .env.local, then restart npm run dev.",
    );
  }
```

Makes the connection. It reads the cookies first; that tells Next.js the page is personal to each visitor, so it is never pre-built during the build step. Then, if the keys are missing, it stops with a clear message saying exactly what to add.

```
  return createServerClient(keys.url, keys.key, {
    cookies: {
      async getAll() {
        return cookieStore.getAll();
      },
      setAll(cookiesToSet) {
        try {
          cookiesToSet.forEach(({ name, value, options }) => cookieStore.set(name, value, options));
        } catch {
          // Called while rendering a page, where cookies can't be set. proxy.ts refreshes them instead.
        }
      },
    },
  });
}
```

Creates the connection, telling it how to read the login cookies and how to save refreshed ones. Saving can fail while a page is being drawn; that's fine, because the doorman (proxy.ts) refreshes them instead.

lib/session.ts

22 lines

Two gatekeepers that every private page and button calls first: "who is this?" and "have they filled in

onboarding?".

```
import { redirect } from "next/navigation";
import { createClient } from "@/lib/supabase/server";
import type { ProfileRow } from "@/lib/types";

/** The signed-in user, or a redirect to /login. Call it in every page and action. */
```

Brings in the redirect tool, the Supabase connection maker and the description of a profile.

```
export async function requireUser() {
  const supabase = await createClient();
  const { data } = await supabase.auth.getClaims();
  const userId = data?.claims?.sub;
  if (!userId) redirect("/login");
  return { supabase, userId };
}

/** Like requireUser, but also needs the onboarding form to be filled in. */
```

Connects to Supabase and asks who is logged in. If nobody, it sends the visitor to /login. Otherwise it hands back the connection and the person's id.

```
export async function requireProfile() {
  const { supabase, userId } = await requireUser();
  const { data, error } = await supabase.from("profiles").select("*").eq("user_id", userId).maybeSingle();
  if (error) throw new Error(error.message);
  if (!data) redirect("/onboarding");
  const profile = { ...data, daily_hours: Number(data.daily_hours) } as ProfileRow;
  return { supabase, userId, profile };
}
```

Does the same, then looks up the person's profile. If there is none yet, it sends them to /onboarding. daily_hours is converted to a plain number for safety.

lib/types.ts — 34 lines

Descriptions of the shape of each kind of data (a profile, a task, a plan item). They store nothing; they let TypeScript warn if code expects a field that doesn't exist.

```
import type { Priority, RepeatRule } from "@/lib/planner";

```

Reuses the Priority and RepeatRule descriptions from the planner so the two files always agree.

```
export type UserType = "student" | "professional" | "other";
export type PlanStatus = "planned" | "done" | "postponed" | "cancelled";

```

The allowed values for user type and plan status, written out so typos are caught.

```
export type ProfileRow = {
  user_id: string;
  user_type: UserType;
  wake_time: string; // "HH:MM:SS" from the database
  sleep_time: string;
  daily_hours: number;
  timezone: string;
};

```

What a profile looks like when it comes back from the database: times arrive as "07:00:00".

```
export type TaskRow = {
  id: string;
  title: string;
  priority: Priority;
  duration_minutes: number;
  deadline: string | null;
  repeat_rule: RepeatRule;
  active: boolean;
};

```

What a task looks like.

```ts
export type PlanItemRow = {
  id: string;
  task_id: string;
  date: string;
  start_time: string;
  end_time: string;
  status: PlanStatus;
  edited: boolean;
  tasks: { title: string; priority: Priority; duration_minutes: number } | null;
};
```

What a plan item looks like, including the title, priority and duration of its task, which the Today page fetches in the same trip.

lib/time.ts

42 lines

Small date and time helpers, so "today" and "now" always mean the person's own time zone rather than the server's (which runs on world time).

```ts
// Dates are "YYYY-MM-DD" and times "HH:MM", always in the user's own time zone.

export function isValidTimeZone(tz: string): boolean {
  try {
    new Intl.DateTimeFormat("en", { timeZone: tz });
    return true;
  } catch {
    return false;
  }
}
```

A note on formats: dates are written 2026-11-09 and times 14:30.

Checks whether a time-zone name such as Asia/Kolkata is real, by trying to use it and seeing if that fails.

```ts
export function localDate(timeZone: string, at = new Date()): string {
  return new Intl.DateTimeFormat("en-CA", { timeZone, year: "numeric", month: "2-digit", day: "2-digit" }).format(at);
}
```

Today's date in a given time zone, in the 2026-11-09 format.

```ts
export function localTime(timeZone: string, at = new Date()): string {
  return new Intl.DateTimeFormat("en-GB", { timeZone, hour: "2-digit", minute: "2-digit", hourCycle: "h23" }).format(at);
}
```

The current time in a given time zone, on the 24-hour clock.

```ts
export function addDays(date: string, days: number): string {
  const d = new Date(`${date}T00:00:00Z`);
  d.setUTCDate(d.getUTCDate() + days);
  return d.toISOString().slice(0, 10);
}
```

Moves a date forwards or backwards by a number of days (used for "the last 7 days").

```ts
export function prettyDate(date: string, style: "long" | "short" = "long"): string {
  return new Intl.DateTimeFormat("en-IN", {
    timeZone: "UTC",
    weekday: style === "long" ? "long" : "short",
    day: "numeric",
    month: style === "long" ? "long" : "short",
  }).format(new Date(`${date}T00:00:00Z`));
}
```

Turns 2026-11-09 into a friendly "Monday, 9 November" (or the short "Mon, 9 Nov").

```ts
export function weekdayName(date: string): string {
  return new Intl.DateTimeFormat("en-IN", { timeZone: "UTC", weekday: "long" }).format(new Date(`${date}T00:00:00Z`));
}
```

Gives just the day name, such as "Monday", used for "Every Monday" on weekly tasks.

```
/** "07:00:00" -> "07:00" */
export function hhmm(time: string): string {
  return time.slice(0, 5);
}
```

Trims "07:00:00" from the database down to "07:00" for display.

lib/planner.ts

159 lines

The brain of "Plan my day". Given your day settings and your tasks, it decides which tasks go into today and at what times. It is a pure calculator: no database, no screen, so it can be tested on its own.

```
// Rule-based "Plan my day" (PRD user story 4).
// Pure function: no database or browser code, so it is easy to test.

```

Notes saying what the file is for.

```
export type Priority = "high" | "medium" | "low";
export type RepeatRule = "none" | "daily" | "weekly";

export type Profile = {
  wake_time: string; // "HH:MM"
  sleep_time: string; // "HH:MM" (may be after midnight, e.g. "01:00")
  daily_hours: number; // max hours of task time per day
};

export type Task = {
  id: string;
  title: string;
  priority: Priority;
  duration_minutes: number;
  deadline: string | null; // "YYYY-MM-DD"
  repeat_rule: RepeatRule;
  active: boolean;
};

```

Descriptions of the inputs: a priority (high, medium, low), a repeat rule (none, daily, weekly), the person's profile (wake time, sleep time, hours available) and a task.

```
export type PlannedItem = {
  task_id: string;
  start_time: string; // "HH:MM"
  end_time: string; // "HH:MM"
};

export type Plan = {
  items: PlannedItem[];
  unscheduled: Task[]; // wanted today but did not fit
};

```

Descriptions of the output: each planned item has a task and a start and end time; a plan is the list of placed items plus the tasks that didn't fit.

```
export const BREAK_MINUTES = 10;
const SLOT_MINUTES = 5;
const PRIORITY_RANK: Record<Priority, number> = { high: 0, medium: 1, low: 2 };

```

Fixed settings: a 10-minute break after every task, start times rounded to 5 minutes, and a ranking where high comes before medium before low.

```
export function toMinutes(hhmm: string): number {
  const [h, m] = hhmm.split(":").map(Number);
  return h * 60 + m;
}

```

Converts a time like 07:30 into minutes since midnight (450), which makes adding and comparing times simple arithmetic.

```
export function toHHMM(minutes: number): string {
  const m = ((minutes % 1440) + 1440) % 1440;
  return `${String(Math.floor(m / 60)).padStart(2, "0")}:${String(m % 60).padStart(2, "0")}`;
}

```

The reverse: turns 450 back into 07:30, wrapping round after midnight.

```
48  function weekday(date: string): number {
49    return new Date(`${date}T00:00:00Z`).getUTCDay();
50  }
51
```

Works out which weekday a date falls on (0 is Sunday).

```
52  /**
53   * Should this task appear on `date`?
54   * - one-time: until it is done (caller passes only unfinished
ones), overdue included
55   * - daily: every day
56   * - weekly: on the same weekday as its deadline (the add-task
form will require one)
57   */
```

Explains the rule below.

```
58  export function isDueOn(task: Task, date: string): boolean {
59    if (!task.active) return false;
60    if (task.repeat_rule === "daily") return true;
61    if (task.repeat_rule === "weekly") {
62      return task.deadline !== null && weekday(task.deadline)
=== weekday(date);
63    }
64    return true;
65  }
66
```

Decides whether a task belongs on a given day. Deleted tasks: never. Daily tasks: every day. Weekly tasks: only on the same weekday as their chosen date. One-time tasks: every day until done (finished ones are filtered out before this).

```
67  // Only one-time tasks have a real deadline; for weekly tasks
the date just picks the weekday.
68  function realDeadline(task: Task): string | null {
69    return task.repeat_rule === "none" ? task.deadline : null;
70  }
71
```

For weekly tasks the date only picks the weekday; it is not a deadline. This helper returns the deadline only for one-time tasks. (Testing caught a mistake here: weekly tasks were being treated as overdue.)

```
72  function compareTasks(a: Task, b: Task, date: string): number
{
73    const deadlineA = realDeadline(a);
74    const deadlineB = realDeadline(b);
75    // 1. Due today or overdue first
76    const urgentA = deadlineA !== null && deadlineA <= date ? 0
: 1;
77    const urgentB = deadlineB !== null && deadlineB <= date ? 0
: 1;
78    if (urgentA !== urgentB) return urgentA - urgentB;
79    // 2. Higher priority first
80    if (a.priority !== b.priority) return
PRIORITY_RANK[a.priority] - PRIORITY_RANK[b.priority];
81    // 3. Earlier deadline first (no deadline last)
82    if (deadlineA !== deadlineB) {
83      if (deadlineA === null) return 1;
84      if (deadlineB === null) return -1;
85      return deadlineA < deadlineB ? -1 : 1;
86    }
87    // 4. Shorter first, to build momentum
88    return a.duration_minutes - b.duration_minutes;
89  }
90
```

The sorting rule used to decide who goes first: 1) tasks due today or overdue, 2) higher priority, 3) earlier deadline, 4) shorter tasks first, to build momentum.

```
91  export type PlanOptions = {
92    nowHHMM?: string; // start no earlier than this
93    busy?: { start_time: string; end_time: string }[]; // time
already taken (kept plan items)
94    usedMinutes?: number; // task minutes already spent today
(counts against daily_hours)
95  };
96
97  /** Minutes since the user's wake-up day began, so times after
midnight sort after evening times. */
```

Extra information the planner can be given: the current time, time already taken by tasks you've finished or moved by hand, and minutes already spent today.

```
98  export function dayMinutes(hhmm: string, wakeHHMM: string):
number {
99    const m = toMinutes(hhmm);
100   return m < toMinutes(wakeHHMM) ? m + 1440 : m;
101 }
102
103 /**
104  * Places tasks one after another from wake time (or from
`nowHHMM` if later),
105  * with a short break after each, until sleep time or the
daily-hours budget runs out.
106  * A task that does not fit is skipped and smaller tasks after
it may still be placed.
107  */
```

Like toMinutes, but treats times after midnight (for night owls who sleep at 1am) as later than evening times rather than earlier.

```
108 export function planDay(profile: Profile, tasks: Task[], date:
string, options: PlanOptions = {}): Plan {
109   const wake = toMinutes(profile.wake_time);
110   let sleep = toMinutes(profile.sleep_time);
111   if (sleep <= wake) sleep += 1440; // sleeping after midnight
112
```

The main calculation begins. It converts wake and sleep times to minutes; if bedtime is after midnight it adds a full day to it.

```
113   let cursor = wake;
114   if (options.nowHHMM !== undefined) {
115     const now = dayMinutes(options.nowHHMM,
profile.wake_time);
116     if (now < sleep) {
117       cursor = Math.max(cursor, Math.ceil(now / SLOT_MINUTES)
* SLOT_MINUTES);
118     } else if (toMinutes(options.nowHHMM) >= wake) {
119       cursor = sleep; // already past bedtime today: nothing
more fits
120     } // otherwise it's the small hours before waking: start
at wake time
121   }
122
```

The "cursor" is the next free moment. It starts at wake time. If you're planning mid-day, it moves to now (rounded up to 5 minutes). If it's already past bedtime, nothing more fits; if it's the small hours before waking, it starts at wake time.

```
123   const busy = (options.busy ?? []).map((b) => {
124     const start = dayMinutes(b.start_time, profile.wake_time);
125     let end = dayMinutes(b.end_time, profile.wake_time);
126     if (end < start) end += 1440;
127     return { start, end: end + BREAK_MINUTES };
128   });
129
```

Turns already-taken time into blocks, each with a break after it, so new tasks don't overlap them.

```
130   let budget = Math.round(profile.daily_hours * 60) -
(options.usedMinutes ?? 0);
131   const items: PlannedItem[] = [];
132   const unscheduled: Task[] = [];
133
134   const due = tasks.filter((t) => isDueOn(t, date)).sort((a,
b) => compareTasks(a, b, date));
135
```

The budget is your daily hours in minutes, minus time already spent. The due tasks are filtered and sorted with the rules above.

```
136   for (const task of due) {
137     // Earliest start at or after the cursor that doesn't
overlap busy time.
138     let start = cursor;
139     for (let moved = true; moved; ) {
140       moved = false;
141       for (const b of busy) {
142         if (start < b.end && start + task.duration_minutes >
b.start) {
143           start = b.end;
144           moved = true;
145         }
146       }
147     }
```

Goes through the sorted tasks one by one. For each, it finds the earliest start at or after the cursor that doesn't clash with a busy block.

```
    const end = start + task.duration_minutes;
    if (task.duration_minutes > budget || end > sleep) {
      unscheduled.push(task);
      continue;
    }
    items.push({ task_id: task.id, start_time: toHHMM(start),
end_time: toHHMM(end) });
    budget -= task.duration_minutes;
    cursor = end + BREAK_MINUTES;
  }

  return { items, unscheduled };
}
```

If the task would go past bedtime or over the budget, it goes on the "didn't fit" list and the next (perhaps smaller) task gets a try. Otherwise it is placed, the budget shrinks, and the cursor jumps past it plus a 10-minute break.

Hands back the finished plan and the list of tasks that didn't fit.

lib/analytics.ts

48 lines

Sends anonymous usage events to PostHog so you can measure the PRD's success metrics. It sends nothing until a PostHog key is added, and it never sends emails or task names.

```
import { after } from "next/server";

// PostHog events (PRD section 9), sent from the server so ad
blockers can't drop them.
// Does nothing until POSTHOG_KEY is set. Only the user's id
is sent: never emails or task names.

export type EventName =
  | "signed up"
  | "signed in"
  | "onboarding completed"
  | "settings updated"
  | "task added"
  | "task edited"
  | "task deleted"
  | "plan my day pressed"
  | "task done"
  | "task postponed"
  | "task cancelled"
  | "task undone"
  | "plan item moved"
  | "plan item removed"
  | "app opened"
  | "account deleted";

export function track(userId: string, event: EventName,
properties: Record<string, unknown> = {}) {
  const key = process.env.POSTHOG_KEY;
  if (!key) return;
  const host = (process.env.POSTHOG_HOST ||
"https://us.i.posthog.com").replace(/\/$/, "");

  // Runs after the response is sent, so tracking never slows
the app down.
```

Brings in after(), a Next.js tool for doing work after the page has already been sent to the visitor. The notes explain the privacy promise.

The full list of events that can be recorded, such as "task done" or "plan my day pressed". Listing them means a misspelt event name is caught before it ships.

The recording function. If there is no PostHog key, it quietly does nothing. Otherwise it picks the PostHog address (US by default).

```
    after(async () => {
      try {
        await fetch(`${host}/i/v0/e/`, {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            api_key: key,
            event,
            distinct_id: userId,
            properties: { ...properties,
$process_person_profile: true },
            timestamp: new Date().toISOString(),
          }),
          signal: AbortSignal.timeout(3000),
        });
      } catch {
        // Analytics must never break the app.
      }
    });
  }
```

After the page is sent, it posts the event to PostHog: the event name, the person's anonymous id, a few details (like how many tasks were planned) and the time. It gives up after 3 seconds and ignores any error, so analytics can never slow down or break the app.

lib/ui.ts

20 lines

The shared "look" for cards, boxes, buttons and labels, so every screen matches. Change it here and every screen changes.

```
  // Shared Tailwind class names so every screen looks the same.
  // Colours come from the palette in app/globals.css (paper,
leaf, ink, saffron, maroon, gold).
  export const card =
    "relative rounded-xl border border-line bg-leaf p-5 shadow-
[0_1px_0_var(--line),0_8px_24px_-16px_color-mix(in_srgb,var(--
maroon)_45%,transparent)]";
```

card: a parchment-coloured panel with a thin border, rounded corners and a soft maroon-tinted shadow.

```
  export const input =
    "w-full rounded-lg border border-line bg-paper/60 px-3 py-2
text-base text-ink outline-none placeholder:text-ink-soft/60
focus:border-saffron focus:ring-2 focus:ring-saffron/20";
```

input: text boxes with a light background that glow saffron when you click into them.

```
  export const label = "flex flex-col gap-1 text-sm font-medium
text-ink-soft";
  export const primaryButton =
    "rounded-lg bg-saffron px-4 py-2 text-sm font-semibold text-
on-accent shadow-sm hover:bg-saffron-deep disabled:opacity-50";
  export const secondaryButton =
    "rounded-lg border border-line bg-leaf px-3 py-1.5 text-sm
font-medium text-ink hover:border-gold hover:bg-leaf-strong
disabled:opacity-50";
```

label: the small brown words above each box. primaryButton: the solid saffron button for the main action. secondaryButton: the outlined button for lesser actions.

```
  export const muted = "text-sm text-ink-soft";
  export const errorText = "text-sm text-maroon";
  export const heading = "font-display text-3xl font-semibold
text-ink";

```

muted: small, softer text for hints. errorText: maroon text for problems. heading: the big page titles in the Eczar typeface.

```
  export const priorityBadge: Record<string, string> = {
    high: "border border-maroon/30 bg-maroon/10 text-maroon",
    medium: "border border-saffron/30 bg-saffron/10 text-
saffron-deep",
    low: "border border-line bg-paper text-ink-soft",
  };
```

The little coloured tags: maroon for high priority, saffron for medium, plain parchment for low.

The look and the shared frame

These files set the colours, typefaces and decorations, and wrap every page in the same frame.

app/globals.css

92 lines

The style sheet: the colour palette, the background texture and a few decorations. CSS is the language browsers use for appearance.

```
1   @import "tailwindcss";
2
3   /* Chanakya palette: palm-leaf parchment, temple gold, saffron
and maroon.
4       Dark mode is a lamp-lit night: deep indigo with gold. */
```

Line 1 switches on Tailwind, the styling system that lets screens use short descriptions like text-ink or rounded-lg.

```
5   :root {
6     --paper: #f3e7cc;
7     --leaf: #fbf4e1;
8     --leaf-strong: #f6eacb;
9     --ink: #3a2213;
10    --ink-soft: #7a5a3c;
11    --line: #dcc596;
12    --gold: #b0802a;
13    --saffron: #c4531a;
14    --saffron-deep: #a3410f;
15    --maroon: #7d1f1f;
16    --on-accent: #fff8ea;
17  }
18
```

The daytime palette, each colour given a name: paper (the parchment background), leaf (cards), ink (text), line (borders), gold, saffron, maroon and on-accent (text on saffron buttons).

```
19  @media (prefers-color-scheme: dark) {
20    :root {
21      --paper: #17111f;
22      --leaf: #221a2e;
23      --leaf-strong: #2b2139;
24      --ink: #f2e5c9;
25      --ink-soft: #c2aa86;
26      --line: #43344f;
27      --gold: #d6aa4f;
28      --saffron: #e9883b;
29      --saffron-deep: #f19c55;
30      --maroon: #e07a6a;
31      --on-accent: #1d1308;
32    }
33  }
34
```

The night palette, used automatically when the phone or computer is in dark mode: a lamp-lit deep indigo with brighter gold and saffron.

```
35  @theme inline {
36    --color-paper: var(--paper);
37    --color-leaf: var(--leaf);
38    --color-leaf-strong: var(--leaf-strong);
39    --color-ink: var(--ink);
40    --color-ink-soft: var(--ink-soft);
41    --color-line: var(--line);
42    --color-gold: var(--gold);
43    --color-saffron: var(--saffron);
44    --color-saffron-deep: var(--saffron-deep);
45    --color-maroon: var(--maroon);
46    --color-on-accent: var(--on-accent);
47    --color-background: var(--paper);
48    --color-foreground: var(--ink);
49    --font-sans: var(--font-mukta);
50    --font-display: var(--font-eczar);
51    --font-mono: var(--font-mukta);
52  }
53
```

Makes those colours available to Tailwind (so code can say bg-saffron) and names the two typefaces: Mukta for text, Eczar for headings.

```
  body {
    background-color: var(--paper);
    /* faint palm-leaf fibres and a warm lamp glow */
    background-image:
      radial-gradient(ellipse 80% 50% at 50% -10%, color-mix(in srgb, var(--saffron) 14%, transparent), transparent 70%),
      repeating-linear-gradient(0deg, color-mix(in srgb, var(--gold) 5%, transparent) 0 1px, transparent 1px 7px);
    background-attachment: fixed;
    color: var(--ink);
    font-family: var(--font-mukta), system-ui, sans-serif;
  }

  ::selection {
    background: color-mix(in srgb, var(--saffron) 30%, transparent);
  }

  /* Temple-border band used at the top of cards and the header */
  .temple-band {
    height: 6px;
    background:
      linear-gradient(135deg, var(--gold) 25%, transparent 25%) -3px 0 / 12px 6px,
      linear-gradient(225deg, var(--gold) 25%, transparent 25%) -3px 0 / 12px 6px,
      var(--maroon);
  }

  @keyframes slow-turn {
    to {
      transform: rotate(360deg);
    }
  }

  .mandala-spin {
    animation: slow-turn 240s linear infinite;
  }

  @media (prefers-reduced-motion: reduce) {
    .mandala-spin {
      animation: none;
    }
  }
```

The page background: parchment, with a warm saffron glow at the top and very faint horizontal lines like the fibres of a palm leaf.

When you highlight text, it turns light saffron.

The patterned strip across the top of pages and cards: gold triangles on a maroon band, like a temple border.

Makes the big mandala on the home page turn very slowly (one turn every 4 minutes). For people whose device asks for reduced motion, it stays still.

app/layout.tsx

28 lines

The outer frame for every single page: it loads the typefaces and sets the page title shown in the browser tab.

```
  import type { Metadata } from "next";
  import { Eczar, Mukta } from "next/font/google";
  import "./globals.css";

```

Brings in the page-information description, the two Google typefaces and the style sheet.

```
  // Eczar and Mukta are typefaces designed in India, for Latin and Devanagari.
  const eczar = Eczar({
    variable: "--font-eczar",
    subsets: ["latin", "devanagari"],
  });

  const mukta = Mukta({
    variable: "--font-mukta",
    subsets: ["latin", "devanagari"],
    weight: ["400", "500", "600", "700"],
  });

```

Loads Eczar (headings) and Mukta (text). Both were designed in India and include Devanagari letters, so चाणक्य displays properly.

```
  export const metadata: Metadata = {
    title: "Chanakya",
    description: "Your personal planner",
  };

```

The tab title ("Chanakya") and the description search engines see.

```
  export default function RootLayout({ children }: LayoutProps<"/">) {
    return (
      <html lang="en" className={`${eczar.variable} ${mukta.variable} h-full antialiased`}>
        <body className="min-h-full flex flex-col">{children}</body>
      </html>
    );
  }
```

The frame itself: the page's language is English, the typefaces are attached, and {children} is the slot where each individual page appears.

components/ornaments.tsx

86 lines

The decorations, drawn with SVG (a way of describing pictures as shapes, so they stay sharp at any size and change colour with the theme).

```
  // Decorative pieces drawn as SVG so they stay sharp and follow the theme colours.

  function petals(count: number, radius: number, length: number, width: number) {
    return Array.from({ length: count }, (_, i) => {
      const angle = (360 / count) * i;
      const tip = radius + length;
      return (
        <path
          key={`${radius}-${i}`}
          transform={`rotate(${angle} 200 200)`}
          d={`M200 ${200 - radius} C ${200 + width} ${200 - radius - length * 0.35}, ${200 + width * 0.6} ${200 - tip + length * 0.2}, 200 ${200 - tip} C ${200 - width * 0.6} ${200 - tip + length * 0.2}, ${200 - width} ${200 - radius - length * 0.35}, 200 ${200 - radius} Z`}
        />
      );
    });
  }

  /** Rangoli-style mandala. */
```

A helper that draws a ring of petals: given how many petals, how far from the centre and how big, it calculates each petal's curved outline and rotates copies evenly around the circle.

```
  export function Mandala({ className = "" }: { className?: string }) {
    return (
      <svg viewBox="0 0 400 400" aria-hidden="true" className={className}>
        <g fill="none" stroke="var(--gold)" strokeWidth="1.2">
          <circle cx="200" cy="200" r="196" strokeDasharray="2 6" />
          <circle cx="200" cy="200" r="186" />
          <circle cx="200" cy="200" r="120" />
          <circle cx="200" cy="200" r="62" />
          <circle cx="200" cy="200" r="30" strokeDasharray="3 4" />
          {petals(32, 120, 62, 9)}
        </g>
        <g fill="color-mix(in srgb, var(--saffron) 14%, transparent)" stroke="var(--saffron)" strokeWidth="1.2">
          {petals(16, 62, 56, 16)}
        </g>
        <g fill="color-mix(in srgb, var(--maroon) 16%, transparent)" stroke="var(--maroon)" strokeWidth="1">
          {petals(8, 30, 30, 12)}
        </g>
        <circle cx="200" cy="200" r="10" fill="var(--gold)" />
      </svg>
    );
  }

  /** Small lotus used in the logo and as list markers. */
```

The mandala: gold circles (two dotted), a ring of 32 slim gold petals, 16 saffron petals, 8 maroon petals and a gold centre.

```
  export function Lotus({ className = "" }: { className?: string }) {
    return (
      <svg viewBox="0 0 48 32" aria-hidden="true" className={className}>
        <g stroke="var(--saffron)" strokeWidth="1.4" fill="color-mix(in srgb, var(--saffron) 18%, transparent)" strokeLinejoin="round">
          <path d="M24 3 C30 10 30 20 24 28 C18 20 18 10 24 3 Z" />
          <path d="M24 28 C20 20 13 15 6 14 C8 22 15 28 24 28 Z" />
          <path d="M24 28 C28 20 35 15 42 14 C40 22 33 28 24 28 Z" />
          <path d="M24 28 C16 26 7 24 2 22 C6 27 14 30 24 28 Z" />
          <path d="M24 28 C32 26 41 24 46 22 C42 27 34 34 30 24 28 Z" />
        </g>
        <path d="M10 30.5 H38" stroke="var(--gold)" strokeWidth="1.4" strokeLinecap="round" />
      </svg>
    );
  }

  /** Ornamental divider: line, dots and a diamond. */
```

The small lotus: five overlapping saffron petals on a gold line, used in the logo and footer.

```tsx
export function Divider({ className = "" }: { className?: string }) {
  return (
    <div aria-hidden="true" className={`flex items-center gap-3 text-gold ${className}`}>
      <span className="h-px flex-1 bg-gradient-to-r from-transparent to-gold/70" />
      <svg viewBox="0 0 60 12" className="h-3 w-14">
        <circle cx="6" cy="6" r="1.6" fill="currentColor" />
        <circle cx="16" cy="6" r="2" fill="currentColor" />
        <path d="M30 0 L36 6 L30 12 L24 6 Z" fill="var(--saffron)" />
        <circle cx="44" cy="6" r="2" fill="currentColor" />
        <circle cx="54" cy="6" r="1.6" fill="currentColor" />
      </svg>
      <span className="h-px flex-1 bg-gradient-to-l from-transparent to-gold/70" />
    </div>
  );
}

/** "चाणक्य Chanakya" wordmark. */
```

The ornamental divider: a fading gold line, dots, a saffron diamond, dots and another fading line.

```tsx
export function Logo({ size = "md" }: { size?: "md" | "lg" }) {
  return (
    <span className="inline-flex items-center gap-2">
      <Lotus className={size === "lg" ? "h-8 w-12" : "h-5 w-8"} />
      <span className="flex flex-col leading-none">
        <span className={`font-display font-semibold text-maroon ${size === "lg" ? "text-2xl" : "text-lg"}`}>
          चाणक्य
        </span>
        <span className="text-[0.65rem] font-semibold uppercase tracking-[0.3em] text-gold">Chanakya</span>
      </span>
    </span>
  );
}
```

The wordmark: the lotus beside चाणक्य in maroon, with CHANAKYA in small, widely spaced gold capitals underneath. It comes in a medium and a large size.

The public pages

Pages anyone can visit without logging in.

app/page.tsx

84 lines

The landing page: the first thing a visitor sees at the web address.

```
1   import Link from "next/link";
2   import { Divider, Logo, Lotus, Mandala } from
"@/components/ornaments";
3   import { card } from "@/lib/ui";
4
```

Brings in the link tool, the decorations and the card style.

```
5   const pillars = [
6     {
7       numeral: "१",
8       title: "Gather",
9       text: "Write down every task with its priority, how long
it takes and when it's due. Repeating duties are written once.",
10    },
11    {
12      numeral: "२",
13      title: "Plan",
14      text: "Press one button. Chanakya places each task in your
free hours, most important first, with rest between.",
15    },
16    {
17      numeral: "३",
18      title: "Reflect",
19      text: "Mark tasks done, postponed or let go. See how much
of your plan you kept, and keep your streak alive.",
20    },
21  ];
22
```

The text of the three steps shown on the page, each with a Devanagari numeral: १ Gather, २ Plan, ३ Reflect.

```
23  export default function Home() {
24    return (
25      <div className="flex flex-1 flex-col">
26        <div className="temple-band" />
27        <header className="mx-auto flex w-full max-w-5xl items-
center justify-between px-4 py-5">
28          <Logo />
29          <Link href="/login" className="text-sm font-semibold
text-ink-soft hover:text-saffron">
30            Sign in
31          </Link>
32        </header>
33
```

The page begins with a temple band and a header holding the logo and a "Sign in" link.

```
        <main className="flex flex-1 flex-col">
          <section className="relative mx-auto flex w-full max-w-5xl flex-col items-center overflow-hidden px-4 pt-10 pb-20 text-center">
            <Mandala className="mandala-spin pointer-events-none absolute top-1/2 left-1/2 -z-10 h-[38rem] w-[38rem] -translate-x-1/2 -translate-y-1/2 opacity-25" />

            <p className="text-sm font-semibold uppercase tracking-[0.35em] text-gold">The art of the well-planned day</p>
            <h1 className="mt-5 max-w-3xl font-display text-5xl leading-tight font-semibold text-ink sm:text-6xl">
              Know what to do, <span className="text-maroon">and when</span>.
            </h1>
            <p className="mt-6 max-w-xl rounded-2xl bg-paper/70 px-3 text-lg leading-8 text-ink-soft backdrop-blur-[2px]">
              Chanakya is your personal planner. Give it your tasks and your hours, and it shapes them into a day you can
              actually follow.
            </p>
            <div className="mt-10 flex flex-wrap justify-center gap-3">
              <Link
                href="/login"
                className="rounded-lg bg-saffron px-7 py-3 font-semibold text-on-accent shadow-md hover:bg-saffron-deep"
              >
                Begin planning
              </Link>
              <a
                href="#how"
                className="rounded-lg border border-gold/60 bg-leaf/70 px-7 py-3 font-semibold text-ink hover:bg-leaf"
              >
                How it works
              </a>
            </div>
          </section>
```

The hero: the slowly turning mandala behind everything, the small gold line "The art of the well-planned day", the big heading "Know what to do, and when.", a short explanation, and two buttons: "Begin planning" (to the login page) and "How it works" (scrolls down).

```
          <section id="how" className="mx-auto w-full max-w-5xl px-4 pb-20">
            <Divider className="mb-10" />
            <h2 className="mb-10 text-center font-display text-3xl font-semibold text-ink">Three steps, every day</h2>
            <div className="grid gap-5 sm:grid-cols-3">
              {pillars.map((p) => (
                <article key={p.title} className={`${card} overflow-hidden pt-8 text-center`}>
                  <div className="temple-band absolute inset-x-0 top-0" />
                  <p className="font-display text-4xl text-gold">{p.numeral}</p>
                  <h3 className="mt-2 font-display text-xl font-semibold text-maroon">{p.title}</h3>
                  <p className="mt-3 leading-7 text-ink-soft">{p.text}</p>
                </article>
              ))}
            </div>
          </section>
        </main>
```

The "Three steps, every day" section: a divider, a heading, and the three step cards, each topped with a temple band.

```
        <footer className="border-t border-line py-8 text-center text-sm text-ink-soft">
          <Lotus className="mx-auto mb-2 h-5 w-8" />
          Plan with purpose. Act with focus.
        </footer>
      </div>
    );
  }
```

The footer: a small lotus and the line "Plan with purpose. Act with focus."

app/error.tsx

16 lines

The friendly page shown if something unexpected goes wrong, instead of a blank screen.

```tsx
"use client";

export default function ErrorPage({ error, retry }: { error: Error; retry: () => void }) {
  return (
    <main className="mx-auto flex w-full max-w-md flex-1 flex-col justify-center gap-4 px-4 py-16">
      <h1 className="font-display text-2xl font-semibold text-maroon">Something went wrong</h1>
      <p className="text-sm text-ink-soft">{error.message}</p>
      <button
        onClick={retry}
        className="self-start rounded-lg border border-line bg-leaf px-3 py-1.5 text-sm font-medium text-ink hover:border-gold"
      >
        Try again
      </button>
    </main>
  );
}
```

"use client" means this part runs in the visitor's browser (it needs a working button).

Shows "Something went wrong", the error message, and a "Try again" button that reloads that part of the page.

app/login/page.tsx

25 lines

The login page's layout: the logo, a line of text and the login box.

```tsx
import { Suspense } from "react";
import { LoginForm } from "./login-form";
import { Logo, Mandala } from "@/components/ornaments";
import { card } from "@/lib/ui";

export default function LoginPage() {
  return (
    <div className="flex flex-1 flex-col">
      <div className="temple-band" />
      <main className="relative mx-auto flex w-full max-w-sm flex-1 flex-col justify-center gap-6 overflow-visible px-4 py-16">
        <Mandala className="pointer-events-none absolute top-1/2 left-1/2 -z-10 h-[30rem] w-[30rem] -translate-x-1/2 -translate-y-1/2 opacity-20" />
        <div className="flex flex-col items-center gap-3 text-center">
          <Logo size="lg" />
          <p className="text-ink-soft">Sign in to plan your day.</p>
        </div>
        <div className={`${card} overflow-hidden pt-7`}>
          <div className="temple-band absolute inset-x-0 top-0" />
          <Suspense>
            <LoginForm />
          </Suspense>
        </div>
      </main>
    </div>
  );
}
```

Brings in the waiting-area tool (Suspense), the login form, the decorations and the card style.

A temple band, a faint mandala behind, the large logo with "Sign in to plan your day.", and a card holding the form. Suspense lets the page appear instantly while the form gets ready.

app/login/login-form.tsx

40 lines

The email and password form itself, with its two buttons and messages.

```tsx
"use client";

import { useActionState } from "react";
import { useSearchParams } from "next/navigation";
import { authenticate } from "./actions";
import { errorText, input, label, muted, primaryButton, secondaryButton } from "@/lib/ui";

export function LoginForm() {
  const [state, action, pending] = useActionState(authenticate, null);
  const confirmFailed = useSearchParams().get("error") === "confirm";

  return (
    <form action={action} className="flex flex-col gap-4">
      <label className={label}>
        Email
        <input name="email" type="email" autoComplete="email" required className={input} />
      </label>
      <label className={label}>
        Password
        <input name="password" type="password" autoComplete="current-password" minLength={6} required className={input} />
      </label>

      {confirmFailed && !state && (
        <p className={errorText}>That confirmation link didn&apos;t work. Try signing in, or sign up again.</p>
      )}
      {state?.error && <p className={errorText}>{state.error}</p>}
      {state?.info && <p className="text-sm font-medium text-gold">{state.info}</p>}

      <div className="flex gap-3">
        <button name="mode" value="signin" disabled={pending} className={`${primaryButton} flex-1`}>
          Sign in
        </button>
        <button name="mode" value="signup" disabled={pending} className={`${secondaryButton} flex-1`}>
          Create account
        </button>
      </div>
      <p className={muted}>New here? Type an email and password, then press Create account.</p>
    </form>
  );
}
```

Runs in the browser. Brings in a React tool for forms that talk to the server, a way to read the address bar, the login action and the shared styles.

Connects the form to the server's authenticate action. state holds any message back from the server; pending is true while waiting. It also checks the address for ?error=confirm (a failed email link).

The Email and Password boxes. The browser itself insists on a valid email and at least 6 characters before sending.

The messages: a note if the email link failed, any error in maroon, and any information (like "check your email") in gold.

Two buttons that send the same form: one labelled to sign in, one to create an account. Both are disabled while waiting, so nobody double-clicks. Below them, a hint for newcomers.

app/login/actions.ts

40 lines

What happens on the server when someone presses "Sign in" or "Create account".

```
"use server";

import { headers } from "next/headers";
import { redirect } from "next/navigation";
import { createClient } from "@/lib/supabase/server";
import { track } from "@/lib/analytics";

```

"use server" means this code only ever runs on the server, never in the browser, so it can be trusted.

```
export type AuthState = { error?: string; info?: string } | null;

```

The shape of the reply sent back to the form: an error, some information, or nothing.

```
export async function authenticate(_prev: AuthState, formData: FormData): Promise<AuthState> {
  const email = String(formData.get("email") ?? "").trim();
  const password = String(formData.get("password") ?? "");
  const mode = formData.get("mode") === "signup" ? "signup" : "signin";

  if (!email || password.length < 6) {
    return { error: "Enter your email and a password of at least 6 characters." };
  }

  const supabase = await createClient();

```

Reads the email, password and which button was pressed. If the email is empty or the password is shorter than 6 characters, it replies with an error straight away.

```
  if (mode === "signup") {
    const origin = (await headers()).get("origin") ?? "";
    const { data, error } = await supabase.auth.signUp({
      email,
      password,
      options: { emailRedirectTo: `${origin}/auth/callback` },
    });
    if (error) return { error: error.message };
    if (data.user) track(data.user.id, "signed up");
    if (!data.session) {
      return { info: "Almost there: open the confirmation link we just emailed you, then sign in." };
    }
```

Creating an account: asks Supabase to make the account, telling it where the confirmation email's link should lead (/auth/callback). It records a "signed up" event. If Supabase wants the email confirmed first, it tells the person to check their inbox.

```
  } else {
    const { data, error } = await supabase.auth.signInWithPassword({ email, password });
    if (error) return { error: error.message };
    track(data.user.id, "signed in");
  }

```

Signing in: checks the email and password with Supabase. Wrong details produce Supabase's error message; correct ones record a "signed in" event.

```
  redirect("/today");
}
```

Success: go to the Today page (which sends new people on to onboarding).

app/auth/callback/route.ts

13 lines

The address the "confirm your email" link opens. It completes the login and sends the person into the app.

```
import { NextResponse, type NextRequest } from "next/server";
import { createClient } from "@/lib/supabase/server";

// The link in the "confirm your email" message lands here.
```

Brings in request and response tools and the Supabase connection maker.

```
5   export async function GET(request: NextRequest) {
6     const code = request.nextUrl.searchParams.get("code");
7     if (code) {
8       const supabase = await createClient();
9       const { error } = await
supabase.auth.exchangeCodeForSession(code);
10      if (!error) return NextResponse.redirect(new URL("/today",
request.url));
11    }
12    return NextResponse.redirect(new URL("/login?error=confirm",
request.url));
13  }
```

Reads the one-time code from the link and swaps it with Supabase for a proper login. If that works, go to /today; if not, back to /login with a note that the link didn't work.

Onboarding

The short form every new person fills in once, so the planner knows their day.

app/onboarding/page.tsx

34 lines

The onboarding page's layout. It also lets people change their answers later (from the Dashboard).

Brings in the tools, the form, the gatekeeper, decorations, styles and the profile description.

```
import { Suspense } from "react";
import { OnboardingForm } from "./onboarding-form";
import { requireUser } from "@/lib/session";
import { Divider, Logo } from "@/components/ornaments";
import { card, muted } from "@/lib/ui";
import type { ProfileRow } from "@/lib/types";

```

The logo, the heading "Tell Chanakya about your day", a divider and a card holding the form. While the person's existing answers load, it shows "Loading…".

```
export default function OnboardingPage() {
  return (
    <div className="flex flex-1 flex-col">
      <div className="temple-band" />
      <main className="mx-auto flex w-full max-w-md flex-1 flex-col justify-center gap-6 px-4 py-16">
        <div className="flex flex-col items-center gap-3 text-center">
          <Logo size="lg" />
          <h1 className="font-display text-2xl font-semibold text-ink">Tell Chanakya about your day</h1>
          <p className="text-ink-soft">This helps it place tasks in hours that work for you.</p>
        </div>
        <Divider />
        <div className={`${card} overflow-hidden pt-7`}>
          <div className="temple-band absolute inset-x-0 top-0" />
          <Suspense fallback={<p className={muted}>Loading…</p>}>
            <ProfileLoader />
          </Suspense>
        </div>
      </main>
    </div>
  );
}

```

Checks who is logged in, fetches their existing answers (if any) and passes them to the form so it can be pre-filled.

```
async function ProfileLoader() {
  const { supabase, userId } = await requireUser();
  const { data } = await supabase.from("profiles").select("*").eq("user_id", userId).maybeSingle();
  return <OnboardingForm profile={data as ProfileRow | null} />;
}
```

app/onboarding/onboarding-form.tsx

60 lines

The form: who you are, when you wake and sleep, and how many hours you can give to tasks.

Runs in the browser and connects to the saveProfile action.

```
"use client";

import { useActionState } from "react";
import { saveProfile } from "./actions";
import { errorText, input, label, primaryButton } from "@/lib/ui";
import type { ProfileRow } from "@/lib/types";

```

```
  export function OnboardingForm({ profile }: { profile:
ProfileRow | null }) {
    const [state, action, pending] = useActionState(saveProfile,
null);

    return (
      <form action={action} className="flex flex-col gap-4">
        <label className={label}>
```

Sets up the form so it can show errors and a "Saving…" state.

```
          I am a
          <select name="user_type" defaultValue=
{profile?.user_type ?? "professional"} className={input}>
            <option value="professional">Working
professional</option>
            <option value="student">Student</option>
            <option value="other">Other</option>
          </select>
        </label>
```

A drop-down: Working professional, Student or Other (pre-selected from earlier answers, otherwise Working professional).

```
        <div className="grid grid-cols-2 gap-4">
          <label className={label}>
            I wake up at
            <input name="wake_time" type="time" required
defaultValue={profile?.wake_time.slice(0, 5) ?? "07:00"}
className={input} />
          </label>
          <label className={label}>
            I go to sleep at
            <input name="sleep_time" type="time" required
defaultValue={profile?.sleep_time.slice(0, 5) ?? "23:00"}
className={input} />
          </label>
        </div>
        <label className={label}>
```

Two time pickers side by side: wake-up (default 07:00) and sleep (default 23:00).

```
          Hours I can give to tasks each day
          <input
            name="daily_hours"
            type="number"
            min={0.5}
            max={24}
            step={0.5}
            required
            defaultValue={profile?.daily_hours ?? 4}
            className={input}
          />
        </label>
```

A number box for available hours, from half an hour to 24, in half-hour steps (default 4).

```
        {/* Filled in by the browser so "today" matches where
you are. */}
        <input
          type="hidden"
          name="timezone"
          defaultValue={profile?.timezone ?? "UTC"}
          ref={(el) => {
            if (el) el.value =
Intl.DateTimeFormat().resolvedOptions().timeZone;
          }}
        />

        {state?.error && <p className={errorText}>{state.error}
</p>}
        <button disabled={pending} className={primaryButton}>
          {pending ? "Saving…" : "Save and continue"}
        </button>
      </form>
    );
  }
```

A hidden field the browser fills with its time zone (for example Asia/Kolkata), so "today" always matches where the person actually is.

Any problem in maroon, and the "Save and continue" button.

app/onboarding/actions.ts

45 lines

Saves the onboarding answers on the server after checking they make sense.

```
"use server";

import { redirect } from "next/navigation";
import { requireUser } from "@/lib/session";
import { isValidTimeZone } from "@/lib/time";
import { toMinutes } from "@/lib/planner";
import { track } from "@/lib/analytics";

```

Server-only code. Brings in the redirect tool, the gatekeeper, the time-zone checker, the minutes helper and analytics.

```
export type ProfileState = { error?: string } | null;

const TIME = /^\d{2}:\d{2}$/;

```

The reply shape (an error or nothing) and a pattern that matches times like 07:00.

```
export async function saveProfile(_prev: ProfileState, formData: FormData): Promise<ProfileState> {
  const { supabase, userId } = await requireUser();

  const userType = String(formData.get("user_type"));
  const wake = String(formData.get("wake_time"));
  const sleep = String(formData.get("sleep_time"));
  const hours = Number(formData.get("daily_hours"));
  const tz = String(formData.get("timezone") ?? "");

```

Checks who is logged in, then reads every answer from the form.

```
  if (!["student", "professional", "other"].includes(userType)) return { error: "Choose what describes you." };
  if (!TIME.test(wake) || !TIME.test(sleep)) return { error: "Enter your wake-up and sleep times." };
  if (wake === sleep) return { error: "Wake-up and sleep times can't be the same." };

```

Sense checks: a valid user type, real times, wake and sleep not the same.

```
  let awake = toMinutes(sleep) - toMinutes(wake);
  if (awake <= 0) awake += 1440;
  if (!(hours > 0) || hours * 60 > awake) {
    return { error: `Available hours must be more than 0 and at most ${Math.floor(awake / 60)} (your waking hours).` };
  }

```

Works out how many hours you are awake (handling bedtimes after midnight) and refuses available hours that are zero or more than your waking hours.

```
  const { data: existing } = await supabase.from("profiles").select("user_id").eq("user_id", userId).maybeSingle();
  const { error } = await supabase.from("profiles").upsert({
    user_id: userId,
    user_type: userType,
    wake_time: wake,
    sleep_time: sleep,
    daily_hours: hours,
    timezone: isValidTimeZone(tz) ? tz : "UTC",
  });

```

Notes whether a profile already exists, then saves the answers (creating or updating), using the browser's time zone if it's valid, otherwise world time.

```
  if (error) return { error: error.message };
  track(userId, existing ? "settings updated" : "onboarding completed", { user_type: userType, daily_hours: hours });

  redirect("/today");
}
```

Records "onboarding completed" (first time) or "settings updated" (later) and goes to Today.

The signed-in app

Everything inside the app once you're logged in: Today, Tasks and Dashboard, plus the actions behind every button.

app/(app)/layout.tsx

35 lines

The frame around every signed-in page: the temple band, the menu bar and the content area. The (app) brackets group these pages without adding anything to the web address.

Brings in the link tool, the sign-out action and the logo.

```
import Link from "next/link";
import { signOut } from "./actions";
import { Logo } from "@/components/ornaments";

```

The three menu items: Today, Tasks and Dashboard.

```
const links = [
  { href: "/today", label: "Today" },
  { href: "/tasks", label: "Tasks" },
  { href: "/dashboard", label: "Dashboard" },
];

```

The frame: a temple band, then a menu bar with the logo (which links to Today), the three menu links, and a "Sign out" button. Below it, the current page appears in a centred column.

```
export default function AppLayout({ children }: LayoutProps<"/">) {
  return (
    <div className="flex flex-1 flex-col">
      <div className="temple-band" />
      <header className="border-b border-line bg-leaf/80 backdrop-blur">
        <nav className="mx-auto flex w-full max-w-2xl flex-wrap items-center gap-x-6 gap-y-2 px-4 py-3">
          <Link href="/today">
            <Logo />
          </Link>
          <div className="flex flex-1 gap-4 text-sm font-semibold">
            {links.map((l) => (
              <Link key={l.href} href={l.href} className="text-ink-soft hover:text-saffron">
                {l.label}
              </Link>
            ))}
          </div>
          <form action={signOut}>
            <button className="text-sm text-ink-soft hover:text-maroon">Sign out</button>
          </form>
        </nav>
      </header>
      <main className="mx-auto flex w-full max-w-2xl flex-col gap-6 px-4 py-8">{children}</main>
    </div>
  );
}
```

app/(app)/actions.ts

190 lines

The server-side work behind every button in the signed-in app: adding, editing and deleting tasks, planning

the day, marking tasks, signing out and deleting the account. Each one checks who is logged in first.

```
"use server";

import { refresh } from "next/cache";
import { redirect } from "next/navigation";
import { requireProfile, requireUser } from "@/lib/session";
import { localDate, localTime } from "@/lib/time";
import { planDay, toHHMM, toMinutes, type Priority, type
RepeatRule } from "@/lib/planner";
import type { PlanItemRow, PlanStatus, TaskRow } from
"@/lib/types";
import { track, type EventName } from "@/lib/analytics";

// ---------- Tasks ----------

```

Server-only code. Brings in tools to refresh the screen and redirect, the gatekeepers, the time helpers, the planner, the data descriptions and analytics.

```
export type TaskState = { error?: string; ok?: boolean } |
null;

```

The reply shape for task forms: an error, or "ok".

```
function readTask(formData: FormData) {
  const title = String(formData.get("title") ?? "").trim();
  const priority = String(formData.get("priority")) as
Priority;
  const duration = Number(formData.get("duration_minutes"));
  const repeat = String(formData.get("repeat_rule")) as
RepeatRule;
  const deadline = String(formData.get("deadline") ?? "") ||
null;

  if (!title) return { error: "Give the task a name." };
  if (!["high", "medium", "low"].includes(priority)) return {
error: "Choose a priority." };
  if (!Number.isInteger(duration) || duration < 5 || duration
> 720) {
    return { error: "Duration must be between 5 and 720
minutes." };
  }
  if (!["none", "daily", "weekly"].includes(repeat)) return {
error: "Choose how often it repeats." };
  if (repeat === "weekly" && !deadline) return { error:
"Weekly tasks need a date: they repeat on that weekday." };

  return {
    task: { title, priority, duration_minutes: duration,
repeat_rule: repeat, deadline: repeat === "daily" ? null :
deadline },
  };
}

```

Reads a task form and checks it: a name is required, priority must be one of three, duration between 5 and 720 minutes, a valid repeat choice, and weekly tasks must have a date. Daily tasks don't keep a date.

```
export async function addTask(_prev: TaskState, formData:
FormData): Promise<TaskState> {
  const { supabase, userId } = await requireUser();
  const parsed = readTask(formData);
  if (!parsed.task) return { error: parsed.error };
  const { error } = await
supabase.from("tasks").insert(parsed.task);
  if (error) return { error: error.message };
  track(userId, "task added", {
    priority: parsed.task.priority,
    repeat_rule: parsed.task.repeat_rule,
    duration_minutes: parsed.task.duration_minutes,
    has_deadline: parsed.task.deadline !== null,
  });
  refresh();
  return { ok: true };
}

```

Checks the form, saves the new task, records a "task added" event (with only its priority, repeat rule, duration and whether it has a deadline: never its name) and refreshes the screen.

```
  export async function updateTask(id: string, _prev: TaskState,
formData: FormData): Promise<TaskState> {
    const { supabase, userId } = await requireUser();
    const parsed = readTask(formData);
    if (!parsed.task) return { error: parsed.error };
    const { error } = await
supabase.from("tasks").update(parsed.task).eq("id",
id).eq("user_id", userId);
    if (error) return { error: error.message };
    track(userId, "task edited");
    redirect("/tasks");
  }

```

Saves changes to an existing task (only if it is yours), records "task edited" and returns to the task list.

```
  /** Hides the task but keeps its history, so the dashboard
stays accurate. */
  export async function deleteTask(id: string) {
    const { supabase, userId } = await requireUser();
    await supabase.from("tasks").update({ active: false
}).eq("id", id).eq("user_id", userId);
    await supabase.from("plan_items").delete().eq("task_id",
id).eq("status", "planned");
    track(userId, "task deleted");
    refresh();
  }

  // ---------- Plan my day ----------

```

Deleting a task actually hides it (active = false) and removes it from any upcoming plans, so the dashboard's history stays accurate. Records "task deleted".

```
  export async function planMyDay() {
    const { supabase, userId, profile } = await
requireProfile();
    const date = localDate(profile.timezone);
    const now = localTime(profile.timezone);

```

"Plan my day" starts: it needs your profile, then works out today's date and the current time where you live.

```
    const [tasksRes, todayRes] = await Promise.all([
      supabase.from("tasks").select("*").eq("user_id",
userId).eq("active", true),
      supabase.from("plan_items").select("*").eq("user_id",
userId).eq("date", date),
    ]);
    if (tasksRes.error) throw new Error(tasksRes.error.message);
    if (todayRes.error) throw new Error(todayRes.error.message);
    const tasks = tasksRes.data as TaskRow[];
    const todays = todayRes.data as Omit<PlanItemRow, "tasks">
[];

```

Fetches, at the same time, all your active tasks and anything already planned today. If either fails, it stops with the error.

```
    // Keep what the user already acted on or moved by hand; re-
plan the rest.
    const kept = todays.filter((i) => i.status !== "planned" ||
i.edited);
    const keptTaskIds = new Set(kept.map((i) => i.task_id));
    const busy = kept.filter((i) => i.status === "planned" ||
i.status === "done");
    const usedMinutes = busy.reduce((sum, i) => {
      const length = toMinutes(i.end_time) -
toMinutes(i.start_time);
      return sum + (length < 0 ? length + 1440 : length);
    }, 0);

```

Keeps what you've already dealt with (done, postponed, cancelled) or moved by hand; everything else will be re-planned. Done and hand-placed items count as busy time and use up part of your daily hours.

```
    // One-time tasks finished on an earlier day don't come back.
    const oneTimeIds = tasks.filter((t) => t.repeat_rule === "none").map((t) => t.id);
    const doneOnce = new Set<string>();
    if (oneTimeIds.length > 0) {
      const { data } = await supabase.from("plan_items").select("task_id").eq("status", "done").in("task_id", oneTimeIds);
      data?.forEach((r) => doneOnce.add(r.task_id));
    }
    const candidates = tasks.filter((t) => !keptTaskIds.has(t.id) && !doneOnce.has(t.id));

```

Finds one-time tasks you finished on an earlier day so they don't come back, and builds the list of tasks to plan.

```
    const plan = planDay(profile, candidates, date, { nowHHMM: now, busy, usedMinutes });

```

Asks the planner (lib/planner.ts) for a plan, starting from now and avoiding busy time.

```
    const del = await supabase
      .from("plan_items")
      .delete()
      .eq("user_id", userId)
      .eq("date", date)
      .eq("status", "planned")
      .eq("edited", false);
    if (del.error) throw new Error(del.error.message);

    if (plan.items.length > 0) {
      const { error } = await supabase.from("plan_items").insert(plan.items.map((i) => ({ ...i, date })));
      if (error) throw new Error(error.message);
    }
```

Removes today's old untouched plan items, then saves the new ones.

```
    track(userId, "plan my day pressed", {
      replan: todays.length > 0,
      planned_count: plan.items.length,
      unscheduled_count: plan.unscheduled.length,
      kept_count: kept.length,
    });
    refresh();
  }

  // ---------- Editing today's plan ----------

```

Records the event with counts (how many planned, how many didn't fit, whether it was a re-plan) and refreshes the screen.

```
  const statusEvent: Record<PlanStatus, EventName> = {
    done: "task done",
    postponed: "task postponed",
    cancelled: "task cancelled",
    planned: "task undone",
  };

```

Matches each status to an event name: done → "task done", and so on. Undo counts as "task undone".

```
  export async function setStatus(id: string, status: PlanStatus) {
    const { supabase, userId } = await requireUser();
    const { error } = await supabase.from("plan_items").update({ status }).eq("id", id).eq("user_id", userId);
    if (!error) track(userId, statusEvent[status]);
    refresh();
  }

```

The Done, Postpone, Cancel and Undo buttons: updates the plan item's status (only if it's yours), records the event and refreshes.

```
export async function moveItem(id: string, formData: FormData) {
  const { supabase, userId } = await requireUser();
  const start = String(formData.get("start_time") ?? "");
  if (!/^\d{2}:\d{2}$/.test(start)) return;
  const { data: item } = await supabase.from("plan_items").select("start_time, end_time").eq("id", id).maybeSingle();
  if (!item) return;
  let length = toMinutes(item.end_time) - toMinutes(item.start_time);
  if (length < 0) length += 1440;
  await supabase
    .from("plan_items")
    .update({ start_time: start, end_time: toHHMM(toMinutes(start) + length), edited: true })
    .eq("id", id)
    .eq("user_id", userId);
  track(userId, "plan item moved");
  refresh();
}
```

The Move button: reads the new start time, keeps the same length, works out the new end time, marks the item as edited by hand, records "plan item moved" and refreshes.

```
export async function removeItem(id: string) {
  const { supabase, userId } = await requireUser();
  await supabase.from("plan_items").delete().eq("id", id).eq("user_id", userId);
  track(userId, "plan item removed");
  refresh();
}
```

"Remove from today": deletes that plan item and records "plan item removed".

```
/** Called once per page load of Today (from the browser, so prefetching doesn't count as a visit). */
export async function recordVisit() {
  const { userId } = await requireUser();
  track(userId, "app opened");
}

// ---------- Account ----------
```

Records an "app opened" event. It's called from the browser so that the app pre-loading a page in the background doesn't count as a real visit.

```
export async function signOut() {
  const { supabase } = await requireUser();
  await supabase.auth.signOut();
  redirect("/login");
}
```

Signs you out with Supabase and goes to the login page.

```
export async function deleteMyAccount(formData: FormData) {
  if (formData.get("confirm") !== "yes") return;
  const { supabase, userId } = await requireUser();
  const { error } = await supabase.rpc("delete_my_account");
  if (error) throw new Error(error.message);
  track(userId, "account deleted");
  await supabase.auth.signOut();
  redirect("/login");
}
```

Only proceeds if the "I understand" box was ticked. Runs the database's delete_my_account action (which erases everything), records the event, signs out and goes to the login page.

app/(app)/today/page.tsx — 176 lines

The Today page: today's plan with times, buttons for each task, and the tasks that didn't fit.

```
1   import Link from "next/link";
2   import { Suspense } from "react";
3   import { moveItem, planMyDay, removeItem, setStatus } from
"../actions";
4   import { VisitTracker } from "./visit-tracker";
5   import { requireProfile } from "@/lib/session";
6   import { hhmm, localDate, prettyDate } from "@/lib/time";
7   import { dayMinutes, isDueOn } from "@/lib/planner";
8   import { card, muted, primaryButton, priorityBadge,
secondaryButton, heading } from "@/lib/ui";
9   import type { PlanItemRow, PlanStatus, TaskRow } from
"@/lib/types";
10
```

Brings in the actions, the visit recorder, the gatekeeper, time helpers, planner helpers, styles and data descriptions.

```
11  export default function TodayPage() {
12    return (
13      <>
14        <VisitTracker />
15        <Suspense fallback={<p className={muted}>Loading your
day…</p>}>
16          <Today />
17        </Suspense>
18      </>
19    );
20  }
21
```

Records a visit, then shows "Loading your day…" while your plan is fetched.

```
22  const statusLabel: Record<PlanStatus, string> = {
23    planned: "",
24    done: "Done",
25    postponed: "Postponed",
26    cancelled: "Cancelled",
27  };
28
```

The words shown for each finished status: Done, Postponed, Cancelled.

```
29  async function Today() {
30    const { supabase, userId, profile } = await
requireProfile();
31    const date = localDate(profile.timezone);
32
33    const [itemsRes, tasksRes] = await Promise.all([
34      supabase.from("plan_items").select("*, tasks(title,
priority, duration_minutes)").eq("user_id", userId).eq("date",
date),
35      supabase.from("tasks").select("*").eq("user_id",
userId).eq("active", true),
36    ]);
37    if (itemsRes.error) throw new Error(itemsRes.error.message);
38    if (tasksRes.error) throw new Error(tasksRes.error.message);
39
```

Needs your profile, works out your today, then fetches today's plan items (with their task details) and your active tasks at the same time.

```
40    const items = (itemsRes.data as PlanItemRow[]).sort(
41      (a, b) => dayMinutes(a.start_time, profile.wake_time) -
dayMinutes(b.start_time, profile.wake_time),
42    );
43    const tasks = tasksRes.data as TaskRow[];
44    const plannedIds = new Set(items.map((i) => i.task_id));
45
```

Sorts the plan by time of day (with after-midnight times last) and notes which tasks are already planned.

```
    // One-time tasks already finished on another day aren't
"left over".
    const oneTimeIds = tasks.filter((t) => t.repeat_rule ===
"none" && !plannedIds.has(t.id)).map((t) => t.id);
    const doneOnce = new Set<string>();
    if (oneTimeIds.length > 0) {
      const { data } = await
supabase.from("plan_items").select("task_id").eq("status",
"done").in("task_id", oneTimeIds);
      data?.forEach((r) => doneOnce.add(r.task_id));
    }
    const leftOver = tasks.filter((t) => !plannedIds.has(t.id)
&& !doneOnce.has(t.id) && isDueOn(t, date));

    const done = items.filter((i) => i.status ===
"done").length;

```

Works out the "left over" tasks: due today but not in the plan, ignoring one-time tasks finished on earlier days. Counts how many are done.

```
    return (
      <>
        <div className="flex flex-wrap items-end justify-between
gap-4">
          <div>
            <p className={muted}>{prettyDate(date)}</p>
            <h1 className={heading}>Today</h1>
            {items.length > 0 && (
              <p className={muted}>
                {done} of {items.length} done
              </p>
            )}
          </div>
          {tasks.length > 0 && (
            <form action={planMyDay}>
              <button className={primaryButton}>{items.length >
0 ? "Re-plan my day" : "Plan my day"}</button>
            </form>
          )}
        </div>

```

The top of the page: the date, the heading "Today", "X of Y done", and the button, which reads "Plan my day" or, if you've already planned, "Re-plan my day".

```
        {tasks.length === 0 && (
          <div className={card}>
            <p>You have no tasks yet.</p>
            <Link href="/tasks" className="mt-2 inline-block
font-medium underline">
              Add your first task
            </Link>
          </div>
        )}

        {tasks.length > 0 && items.length === 0 && (
          <p className={muted}>Press &ldquo;Plan my day&rdquo;
and Chanakya will fit your tasks into today&apos;s free hours.</p>
        )}

```

If you have no tasks yet, a card invites you to add your first one. If you have tasks but haven't planned, a hint explains the button.

```
        {items.length > 0 && (
          <ol className="flex flex-col gap-2">
            {items.map((item) => (
              <PlanItem key={item.id} item={item} />
            ))}
          </ol>
        )}

```

The plan itself: a numbered list of plan items, each drawn by PlanItem below.

```
        {leftOver.length > 0 && (
          <section className="flex flex-col gap-2">
            <h2 className="font-display text-lg font-semibold
text-maroon">Not in today&apos;s plan</h2>
            <p className={muted}>
              {items.length > 0
                ? "These didn't fit in your hours today, or were
added after you planned."
                : "These tasks are waiting to be planned."}
            </p>
            <ul className="flex flex-col gap-1">
              {leftOver.map((t) => (
                <li key={t.id} className="text-sm">
                  {t.title} <span className={muted}>·
{t.duration_minutes} min</span>
                </li>
              ))}
            </ul>
          </section>
        )}
      </>
    );
}

function PlanItem({ item }: { item: PlanItemRow }) {
  const finished = item.status !== "planned";
  return (
    <li className={`${card} flex flex-col gap-3 !p-4
${finished ? "opacity-60" : ""}`}>
        <div className="flex items-start gap-3">
          <p className="w-28 shrink-0 font-display text-sm font-
semibold text-gold tabular-nums">
            {hhmm(item.start_time)}-{hhmm(item.end_time)}
          </p>
          <p className={`flex-1 font-medium ${item.status ===
"done" ? "line-through" : ""}`}>
            {item.tasks?.title ?? "Deleted task"}
          </p>
          {item.tasks && (
            <span className={`rounded-full px-2 py-0.5 text-xs
font-medium ${priorityBadge[item.tasks.priority]}`}>
              {item.tasks.priority}
            </span>
          )}
        </div>
```

"Not in today's plan": the tasks that didn't fit your hours, or were added after you planned.

One plan item. Finished ones are faded.

Shows the time in gold, the task name (crossed out when done) and its priority tag.

```
      {finished ? (
        <div className="flex items-center gap-3">
          <span className="text-sm font-medium">
{statusLabel[item.status]}</span>
          <form action={setStatus.bind(null, item.id,
"planned")}>
            <button className={secondaryButton}>Undo</button>
          </form>
        </div>
      ) : (
        <div className="flex flex-wrap items-center gap-2">
          <form action={setStatus.bind(null, item.id,
"done")}>
            <button className={primaryButton}>Done</button>
          </form>
          <form action={setStatus.bind(null, item.id,
"postponed")}>
            <button className=
{secondaryButton}>Postpone</button>
          </form>
          <form action={setStatus.bind(null, item.id,
"cancelled")}>
            <button className=
{secondaryButton}>Cancel</button>
          </form>
          <details className="ml-auto text-sm">
            <summary className="cursor-pointer text-ink-soft
hover:text-saffron">Change</summary>
            <div className="mt-2 flex flex-wrap items-center
gap-2">
              <form action={moveItem.bind(null, item.id)}
className="flex items-center gap-2">
                <input
                  type="time"
                  name="start_time"
                  defaultValue={hhmm(item.start_time)}
                  required
                  className="rounded-lg border border-line bg-
paper/60 px-2 py-1 text-ink"
                />
                <button className=
{secondaryButton}>Move</button>
              </form>
              <form action={removeItem.bind(null, item.id)}>
                <button className={secondaryButton}>Remove
from today</button>
              </form>
            </div>
          </details>
        </div>
      )}
    </li>
  );
}
```

If finished: the status and an Undo button. If not: Done, Postpone and Cancel buttons, and a fold-out "Change" menu with a time picker to move the task and a "Remove from today" button.

app/(app)/today/visit-tracker.tsx

19 lines

A tiny invisible helper that records "app opened" at most once a day per device, to measure the PRD's 7-day return rate.

```
"use client";

import { useEffect } from "react";
import { recordVisit } from "../actions";

/** Records "app opened" at most once per day per device (for
the 7-day return rate). */
```

Runs in the browser; uses an effect (code that runs after the page appears) and the record-visit action.

```
  export function VisitTracker() {
    useEffect(() => {
      const today = new Date().toLocaleDateString("en-CA");
      try {
        if (localStorage.getItem("chanakya:last-visit") === today) return;
        localStorage.setItem("chanakya:last-visit", today);
      } catch {
        // Storage blocked (private mode): record anyway.
      }
      recordVisit().catch(() => {});
    }, []);
    return null;
  }
```

Checks the browser's own small storage for today's date. If it's already there, it does nothing; otherwise it writes today's date and records the visit. If storage is blocked (private browsing), it records anyway. It draws nothing on screen.

app/(app)/tasks/page.tsx

67 lines

The Tasks page: a form to add a task, and the list of all your tasks.

```
  import Link from "next/link";
  import { Suspense } from "react";
  import { addTask, deleteTask } from "../actions";
  import { TaskForm } from "./task-form";
  import { requireUser } from "@/lib/session";
  import { prettyDate, weekdayName } from "@/lib/time";
  import { card, muted, priorityBadge, secondaryButton, heading } from "@/lib/ui";
  import type { TaskRow } from "@/lib/types";

```

Brings in the link tool, waiting-area tool, actions, the task form, the gatekeeper, date helpers, styles and the task description.

```
  export default function TasksPage() {
    return (
      <>
        <h1 className={heading}>Tasks</h1>
        <section className={card}>
          <h2 className="mb-4 font-display text-lg font-semibold text-maroon">Add a task</h2>
          <TaskForm action={addTask} submitLabel="Add task" />
        </section>
        <Suspense fallback={<p className={muted}>Loading your tasks…</p>}>
          <TaskList />
        </Suspense>
      </>
    );
  }

```

The heading, a card with "Add a task" and the form, then the task list (with "Loading your tasks…" while it loads).

```
  function repeatText(t: TaskRow) {
    if (t.repeat_rule === "daily") return "Every day";
    if (t.repeat_rule === "weekly" && t.deadline) return `Every ${weekdayName(t.deadline)}`;
    return t.deadline ? `Due ${prettyDate(t.deadline, "short")}` : "No deadline";
  }

```

The small description under each task: "Every day", "Every Monday", "Due Mon, 9 Nov" or "No deadline".

```
async function TaskList() {
  const { supabase, userId } = await requireUser();
  const { data, error } = await supabase
    .from("tasks")
    .select("*")
    .eq("user_id", userId)
    .eq("active", true)
    .order("created_at", { ascending: false });
  if (error) throw new Error(error.message);
  const tasks = data as TaskRow[];

  if (tasks.length === 0) {
    return <p className={muted}>No tasks yet. Add your first
one above.</p>;
  }

  return (
```

Fetches your active tasks, newest first. If there are none, says so.

```
    <ul className="flex flex-col gap-2">
      {tasks.map((t) => (
        <li key={t.id} className={`${card} flex items-center
gap-3 !p-4`}>
          <div className="flex-1">
            <p className="font-medium">{t.title}</p>
            <p className={muted}>
              {t.duration_minutes} min · {repeatText(t)}
            </p>
          </div>
          <span className={`rounded-full px-2 py-0.5 text-xs
font-medium ${priorityBadge[t.priority]}`}>{t.priority}</span>
          <Link href={`/tasks/${t.id}`} className=
{secondaryButton}>
            Edit
          </Link>
          <form action={deleteTask.bind(null, t.id)}>
            <button className=
{secondaryButton}>Delete</button>
          </form>
        </li>
      ))}
    </ul>
  );
}
```

Each task in a card: its name, duration and repeat description, its priority tag, an Edit link and a Delete button.

app/(app)/tasks/task-form.tsx

72 lines

The task form, shared by "Add a task" and "Edit task". Only the name is required, so adding a task takes seconds.

```
"use client";

import { useActionState, useState } from "react";
import type { TaskState } from "../actions";
import { errorText, input, label, primaryButton } from
"@/lib/ui";
import type { TaskRow } from "@/lib/types";

```

Runs in the browser. Brings in form tools, the reply shape, styles and the task description.

```
type Props = {
  action: (prev: TaskState, formData: FormData) =>
Promise<TaskState>;
  task?: TaskRow;
  submitLabel: string;
};

```

What the form needs: which action to send to (add or update), optionally the task being edited, and the button's label.

```
14  export function TaskForm({ action, task, submitLabel }: Props) {
15    const [state, formAction, pending] = useActionState(action, null);
16    const [repeat, setRepeat] = useState(task?.repeat_rule ?? "none");
17
18    return (
19      <form action={formAction} className="flex flex-col gap-4">
20        <label className={label}>
```

Connects to the action and remembers the repeat choice, because the date box changes depending on it.

```
21          Task
22          <input name="title" required autoFocus={!task} defaultValue={task?.title} placeholder="e.g. Finish weekly report" className={input} />
23        </label>
```

The task name box (focused automatically when adding).

```
24        <div className="grid grid-cols-2 gap-4 sm:grid-cols-4">
25          <label className={label}>
26            Priority
27            <select name="priority" defaultValue={task?.priority ?? "medium"} className={input}>
28              <option value="high">High</option>
29              <option value="medium">Medium</option>
30              <option value="low">Low</option>
31            </select>
32          </label>
33          <label className={label}>
34            Minutes
35            <input
36              name="duration_minutes"
37              type="number"
38              min={5}
39              max={720}
40              step={5}
41              required
42              defaultValue={task?.duration_minutes ?? 30}
43              className={input}
44            />
45          </label>
46          <label className={label}>
47            Repeats
48            <select
49              name="repeat_rule"
50              value={repeat}
51              onChange={(e) => setRepeat(e.target.value as typeof repeat)}
52              className={input}
53            >
54              <option value="none">Once</option>
55              <option value="daily">Daily</option>
56              <option value="weekly">Weekly</option>
57            </select>
58          </label>
```

A row of choices: Priority (default Medium), Minutes (default 30, steps of 5), Repeats (Once, Daily, Weekly).

```
59          {repeat !== "daily" && (
60            <label className={label}>
61              {repeat === "weekly" ? "Repeats on" : "Deadline"}
62              <input name="deadline" type="date" required={repeat === "weekly"} defaultValue={task?.deadline ?? ""} className={input} />
63            </label>
64          )}
65        </div>
```

The date box: called "Deadline" for one-time tasks (optional) and "Repeats on" for weekly tasks (required). Daily tasks don't need one, so it's hidden.

```
66        {state?.error && <p className={errorText}>{state.error}
</p>}
67        <button disabled={pending} className={`${primaryButton}
self-start`}>
68          {pending ? "Saving…" : submitLabel}
69        </button>
70      </form>
71    );
72  }
```

Any problem in maroon, and the save button (which reads "Saving…" while working).

app/(app)/tasks/[id]/page.tsx

32 lines

The Edit task page. The [id] in the folder name means the address includes which task, such as /tasks/abc-123.

```
1   import Link from "next/link";
2   import { notFound } from "next/navigation";
3   import { Suspense } from "react";
4   import { updateTask } from "../../actions";
5   import { TaskForm } from "../task-form";
6   import { requireUser } from "@/lib/session";
7   import { card, muted, heading } from "@/lib/ui";
8   import type { TaskRow } from "@/lib/types";
9
```

Brings in the tools, the update action, the task form, the gatekeeper, styles and the task description.

```
10  export default function EditTaskPage({ params }:
PageProps<"/tasks/[id]">) {
11    return (
12      <>
13        <Link href="/tasks" className={muted}>
14          ← Back to tasks
15        </Link>
16        <h1 className={heading}>Edit task</h1>
17        <section className={card}>
18          <Suspense fallback={<p className={muted}>Loading…
</p>}>
19            <EditTask params={params} />
20          </Suspense>
21        </section>
22      </>
23    );
24  }
25
```

A "← Back to tasks" link, the heading and a card with the form.

```
26  async function EditTask({ params }: { params: Promise<{ id:
string }> }) {
27    const { id } = await params;
28    const { supabase, userId } = await requireUser();
29    const { data } = await
supabase.from("tasks").select("*").eq("id", id).eq("user_id",
userId).eq("active", true).maybeSingle();
30    if (!data) notFound();
31    return <TaskForm action={updateTask.bind(null, id)} task=
{data as TaskRow} submitLabel="Save changes" />;
32  }
```

Reads which task from the address, fetches it (only if it's yours and not deleted), shows a "not found" page if it doesn't exist, and otherwise shows the form pre-filled.

app/(app)/dashboard/page.tsx

119 lines

The Dashboard: today's completion, your streak, the last 7 days, and settings (including deleting your account).

```
1   import Link from "next/link";
2   import { Suspense } from "react";
3   import { deleteMyAccount } from "../actions";
4   import { requireProfile } from "@/lib/session";
5   import { addDays, localDate, prettyDate } from "@/lib/time";
6   import { card, muted, secondaryButton, heading } from
"@/lib/ui";
7
```

Brings in the tools, the delete-account action, the gatekeeper, date helpers and styles.

```
  export default function DashboardPage() {
    return (
      <>
        <h1 className={heading}>Dashboard</h1>
        <Suspense fallback={<p className={muted}>Loading your progress…</p>}>
          <Stats />
        </Suspense>
        <Settings />
      </>
    );
  }

  function percent(done: number, total: number) {
    return total === 0 ? "–" : `${Math.round((done / total) * 100)}%`;
  }

  async function Stats() {
    const { supabase, userId, profile } = await requireProfile();
    const today = localDate(profile.timezone);
    const since = addDays(today, -59); // enough history for a long streak

    const { data, error } = await supabase
      .from("plan_items")
      .select("date, status")
      .eq("user_id", userId)
      .gte("date", since)
      .lte("date", today);
    if (error) throw new Error(error.message);

    const byDate = new Map<string, { done: number; total: number }>();
    for (const row of data as { date: string; status: string }[]) {
      const day = byDate.get(row.date) ?? { done: 0, total: 0 };
      day.total += 1;
      if (row.status === "done") day.done += 1;
      byDate.set(row.date, day);
    }

    const todayStats = byDate.get(today) ?? { done: 0, total: 0 };
    const week = Array.from({ length: 7 }, (_, i) => addDays(today, i - 6));
    const weekDone = week.reduce((s, d) => s + (byDate.get(d)?.done ?? 0), 0);
    const weekTotal = week.reduce((s, d) => s + (byDate.get(d)?.total ?? 0), 0);

    // Streak: days in a row with at least one task done. Today only counts once something is done.
    let streak = 0;
    let day = (byDate.get(today)?.done ?? 0) > 0 ? today : addDays(today, -1);
    while ((byDate.get(day)?.done ?? 0) > 0) {
      streak += 1;
      day = addDays(day, -1);
    }
```

The heading, the stats (with "Loading your progress…" while they load) and the settings card.

Turns "3 done out of 5" into 60%, or a dash when nothing was planned.

Needs your profile, then fetches the date and status of every plan item from the last 60 days (enough to measure a long streak).

Groups them by day, counting how many were planned and how many done.

Works out today's numbers and the totals for the last 7 days.

The streak counts days in a row with at least one task done, going backwards. Today only counts once something is done, so the streak doesn't look broken first thing in the morning.

```
    return (
      <>
        <div className="grid grid-cols-1 gap-3 sm:grid-cols-3">
          <Stat label="Done today" value=
{percent(todayStats.done, todayStats.total)} note=
{`${todayStats.done} of ${todayStats.total} tasks`} />
          <Stat label="Streak" value={`${streak} ${streak === 1
? "day" : "days"}`} note="days in a row with a task done" />
          <Stat label="Last 7 days" value={percent(weekDone,
weekTotal)} note={`${weekDone} of ${weekTotal} planned tasks
done`} />
        </div>

        <section className={card}>
          <h2 className="mb-4 font-display text-lg font-semibold
text-maroon">Your last 7 days</h2>
          <ul className="flex flex-col gap-2">
            {week.map((d) => {
              const s = byDate.get(d) ?? { done: 0, total: 0 };
              const width = s.total === 0 ? 0 : (s.done /
s.total) * 100;
              return (
                <li key={d} className="flex items-center gap-3
text-sm">
                  <span className="w-28 shrink-0">{prettyDate(d,
"short")}</span>
                  <span className="h-2.5 flex-1 overflow-hidden
rounded-full border border-line bg-paper">
                    <span className="block h-full rounded-full
bg-gradient-to-r from-gold to-saffron" style={{ width: `${width}%`
}} />
                  </span>
                  <span className="w-12 shrink-0 text-right
text-ink-soft tabular-nums">
                    {s.done}/{s.total}
                  </span>
                </li>
              );
            })}
          </ul>
        </section>
      </>
    );
  }

```

Three stat cards (Done today, Streak, Last 7 days), then a card listing each of the last 7 days with a gold-to-saffron bar showing how much was done.

```
  function Stat({ label, value, note }: { label: string; value:
string; note: string }) {
    return (
      <div className={card}>
        <p className={muted}>{label}</p>
        <p className="font-display text-3xl font-semibold text-
maroon">{value}</p>
        <p className={muted}>{note}</p>
      </div>
    );
  }

```

One stat card: a small label, a big number in maroon and a note.

```
function Settings() {
  return (
    <section className={`${card} flex flex-col gap-4`}>
      <h2 className="font-display text-lg font-semibold text-maroon">Settings</h2>
      <Link href="/onboarding" className={`${secondaryButton} self-start`}>
        Change wake, sleep and available hours
      </Link>
      <form action={deleteMyAccount} className="flex flex-col gap-2 border-t border-line pt-4">
        <p className="text-sm font-medium">Delete my account</p>
        <p className={muted}>This permanently removes your account, tasks and plans. It can&apos;t be undone.</p>
        <label className="flex items-center gap-2 text-sm">
          <input type="checkbox" name="confirm" value="yes" required /> I understand, delete everything
        </label>
        <button className="self-start rounded-lg bg-maroon px-4 py-2 text-sm font-semibold text-on-accent hover:opacity-90">
          Delete my account
        </button>
      </form>
    </section>
  );
}
```

Settings: a link to change your day settings (the onboarding form), and "Delete my account" with a warning, a required "I understand" tick box and a maroon button.